Projector-Generated Special Functions VI

Meixner Shape Semigroups, Beta–Binomial Packet Refinement, Polynomial Martingales, and Gamma–Sech Berezin Kernels

Komorebi P.

Abstract

This paper develops the shape coordinate that was implicit in the log-Gamma and Meixner structures of the preceding paper. For every $\alpha > 0$, define the normalized log-Gamma packet

$$g_\alpha(y) = \Gamma(\alpha)^{-1/2} \exp\left(\frac{\alpha y}{2} - \frac{\mathrm{e}^y}{2}\right).$$

Two packets of different shape have the exact cross-ambiguity function

$$\boxed{\mathcal{A}_{\alpha,\beta}(h,\eta) = \mathrm{e}^{(\alpha-\beta)h/4} \frac{\Gamma((\alpha+\beta)/2 + \mathrm{i}\eta)}{\sqrt{\Gamma(\alpha)\Gamma(\beta)}} \operatorname{sech}^{(\alpha+\beta)/2+\mathrm{i}\eta}\left(\frac{h}{2}\right).}$$

Its squared modulus factors exactly into a beta-logit displacement density and a symmetric Meixner spectral density. This factorization exposes a continuous convolution semigroup in the shape parameter. The semigroup has an explicit Lévy density, a nonlocal generator, and even cumulants governed by Bernoulli numbers.

The orthogonal-polynomial side carries an exact refinement law. A Meixner–Pollaczek packet of total shape $\alpha+\beta$ splits into products of packets of shapes α and β. The squared coefficients form a beta-binomial distribution and sum to one. The many-child extension is governed by a Dirichlet-multinomial law and is coassociative. Under the corresponding Meixner increment process, the unnormalized packet polynomials are martingales. In orthonormal coordinates, the transfer operator has the exact singular values

$$\boxed{\sigma_n(\alpha,t) = \sqrt{\frac{(\alpha)_n}{(\alpha+t)_n}}.}$$

These singular values satisfy a cocycle law and give immediate compactness and Schatten-class criteria.

The Weyl translates of g_α form a continuous tight frame. Their overlap is the Gamma–sech kernel above, and their coherent-state transform is an isometry into a reproducing-kernel Hilbert space. The associated Berezin transform is convolution with a probability kernel that factors into independent position and frequency marginals. Its Fourier multiplier is strictly positive, so the transform is injective, while its decay shows that the inverse is unbounded. After the natural shape rescaling, both marginals converge to standard Gaussians and the Berezin transform converges to the two-dimensional heat operator $\mathrm{e}^{\Delta/2}$.

The classical ingredients include log-Gamma laws, Meixner distributions, Meixner–Pollaczek polynomials, coherent states, and Berezin quantization. The generated contribution is the exact packet system joining cross-shape ambiguity, shape convolution, beta-binomial refinement, polynomial transfer, coherent frames, and the Gaussian fixed point in one calculus. The paper does not prove the signed Möbius coherence estimate and does not prove the Riemann hypothesis.

All figures were generated by the accompanying Python program. The release contains 28 vector figures, matching PNG files, exact tables, 24 built-in checks, independent tests, and the complete Python source.

Status and claim boundary

The paper proves the cross-shape Gamma–sech formula, the shape-affinity inequality, the beta-logit and Meixner factorization, the convolution-semigroup law, the Lévy–Khintchine representation, the Bernoulli cumulant formula, the normalized two-child and many-child packet theorems, coassociativity, the polynomial martingale law, the complete singular-value formula, the coherent-state resolution, the reproducing kernel, the exact Berezin factorization, its strictly positive multiplier, and the Gaussian heat limit.

The words generated and packet refer to the way these classical objects arise from one exact refinement system. They do not assert that the Gamma function, the Meixner law, the beta-binomial distribution, the Meixner–Pollaczek polynomials, or coherent-state quantization are historically new. The candidate contribution is the common normalization and the collection of exact transfer laws linking them.

The quantum discussion uses the logarithmic number-basis coordinate developed in the 2026 construction of Wei, Zhai, Lu, and collaborators. That external work realizes phases $n^{-\mathrm{i}t}$ through a logarithmic Hamiltonian. The present shape semigroup and coherent frame are different observables and no figure from that article is reproduced or adapted.

The positive shape theory does not control the signed principal Möbius packet. None of the theorems below establishes the remaining coherence bound required for RH.

Contents

Status and claim boundary i

I Cross-shape log-Gamma packets 1

1 The shape coordinate 1

2 Cross-shape ambiguity 3

II Factorization and the shape semigroup 5

3 Beta-logit displacement and Meixner frequency 5

4 Convolution in the shape parameter 7

5 Lévy generator and Bernoulli cumulants 9

III Polynomial packets and exact refinement 11

6 Meixner–Pollaczek packet coordinates 11

7 Many-child refinement and coassociativity 14

IV Polynomial martingales and shape transfer 16

8 The Meixner increment process 16

V Coherent frames and Berezin quantization 19

9 Log-Gamma coherent states 19

10 The Gamma–sech Berezin kernel 21

11 Gaussian heat fixed point 23

VI Geometry, implementation, and research boundary 25

12 Three exact local scales 25

13 Relation to the arithmetic packet series 27

14 Quantum coordinate and implementation boundary 28

15 Open problems native to the shape calculus 28

16 Verification 29

A Formula registry **30**

B Selected numerical tables **30**

C Complete Python implementation **31**

D Reproduction **46**

E Bibliographic note **46**

List of Figures

1.1 From the shape parameter to a complete packet system. The upper row follows the exact mathematical chain. The lower row records two consequences of the semigroup structure: Bernoulli cumulants and the Gaussian heat fixed point. . . . 2
1.2 Log-Gamma packets at four shape values. Changing α translates and narrows the active logarithmic window. The normalization remains one in $L^2(\mathbb{R})$. . . . 2
2.1 Magnitude of one cross-shape ambiguity function. The packet remains localized in both translation and modulation. Unequal shapes tilt the displacement law but do not change the analytic form of the frequency factor. . . . 4
2.2 Affinity across the two-dimensional shape plane. The diagonal is the unique unit-overlap set. The curvature transverse to the diagonal is controlled by the trigamma function. . . . 5
3.1 The displacement marginal. The transformation $u = e^h/(1 + e^h)$ converts each curve into a beta density. Exchanging α and β reflects the packet through the origin. . . . 6
3.2 The symmetric Meixner packet family. The shape parameter is additive under convolution. Larger shape produces a broader spectral law because its variance is $s/4$. . . . 7
3.3 Direct and factorized cross-packet energy. The first two panels are the direct and product constructions. Their difference is at floating-point roundoff. . . . 7
4.1 Successive densities generated by the Gamma recurrence. Multiplying by a quadratic polynomial and renormalizing raises the shape by two. . . . 8
4.2 A direct convolution check and its residual. The convolution of shapes 1.3 and 2.2 matches the shape 3.5 density across the plotted window. . . . 9
5.1 The Lévy density and generator symbol. The singularity at the origin is canceled by the second difference in the generator. The numerical integral agrees with the exact Fourier symbol. . . . 10
5.2 The first four even cumulants as shape varies. Every cumulant is linear in the shape parameter. The coefficients are exact Bernoulli-number combinations. . . . 11
6.1 The exact two-child polynomial splitter. One parent packet is reconstructed from every degree allocation across the two children. The beta-binomial coefficient is the exact transfer amplitude. . . . 12
6.2 Beta-binomial packet energy for several shape pairs. The squared coefficients form probability laws. Symmetric shapes produce symmetric degree allocation, while unequal shapes tilt the energy toward one child. . . . 13
6.3 A worked degree-four reconstruction. The left panel overlays the parent and child synthesis. The right panel shows the pointwise floating-point residual. . . . 14
7.1 Three-child packet allocation on a degree-eight simplex. The marker area and color encode squared transfer amplitude. The complete coefficient energy is one. . . . 15
7.2 Two refinement trees with one terminal coefficient. The coefficient does not depend on bracketing. This is the exact associativity law needed for arbitrary packet trees. . . . 16
8.1 Exact singular values across polynomial degree. Large shape increments contract high-degree modes more strongly. The constant mode is preserved exactly. . . . 17
8.2 The contraction spectrum as a function of initial shape and degree. The packet transfer is diagonal in the orthonormal polynomial basis. . . . 18
8.3 The first polynomial martingale factors. Each degree has one exact contraction coefficient. The transfer law is finite, diagonal, and compatible with repeated shape increments. . . . 19
9.1 A finite sample of the coherent-state orbit. Marker size is the overlap magnitude with the packet at the origin. The full orbit is continuous and resolves the identity. . . . 20
9.2 Magnitude of the reproducing kernel. The kernel is the equal-shape Gamma–sech ambiguity. It is localized in both phase-space directions and is nonzero on the real plane. . . . 21
10.1 The two marginal laws and their product. The smoothing kernel is exactly separable in the natural position and frequency coordinates. . . . 22
10.2 The positive multiplier on dual phase space. The transform loses no frequency exactly, but high dual frequencies are strongly suppressed. This explains injectivity together with unstable inversion. . . . 23

11.1 The two standardized marginals approaching the same normal density. The convergence is already close at moderate shape and becomes visually indistinguishable by $\alpha = 80$. 24
11.2 The rescaled Berezin kernel, the Gaussian heat kernel, and their residual. The packet quantization has a classical heat-kernel fixed point in the large-shape limit. 25
12.1 The three diagonal metric coefficients. Shape and modulation sensitivity decrease with α, while translation sensitivity increases. The large-shape packet is narrow in position and broad in frequency. 26
12.2 The exact hierarchy proved in this paper. Shape convolution generates the polynomial refinement system. Coherent-state quantization converts the same kernel into a positive phase-space smoothing operator. . . . 26
13.1 The generated function-family map. Each lower object is a proved consequence of one of the upper transfer laws. The map records dependencies rather than a historical novelty claim. 27

Part I

Cross-shape log-Gamma packets

Chapter 1

The shape coordinate

The log-Gamma packet is

$$\boxed{g_\alpha(y) = \Gamma(\alpha)^{-1/2} \mathrm{e}^{\alpha y/2 - \mathrm{e}^y/2}, \qquad \alpha > 0.}$$

Its squared modulus is the density of $Y = \log U$ when U has the Gamma law with shape α and unit scale.

Proposition 1.1 (Normalization)**.** *For every $\alpha > 0$,*

$$\int_{\mathbb{R}} |g_\alpha(y)|^2 \, \mathrm{d}y = 1.$$

Proof. Set $u = \mathrm{e}^y$. Then $\mathrm{d}y = \mathrm{d}u/u$, and

$$\int_{\mathbb{R}} |g_\alpha(y)|^2 \, \mathrm{d}y = \frac{1}{\Gamma(\alpha)} \int_0^\infty u^{\alpha-1} \mathrm{e}^{-u} \, \mathrm{d}u = 1.$$

□

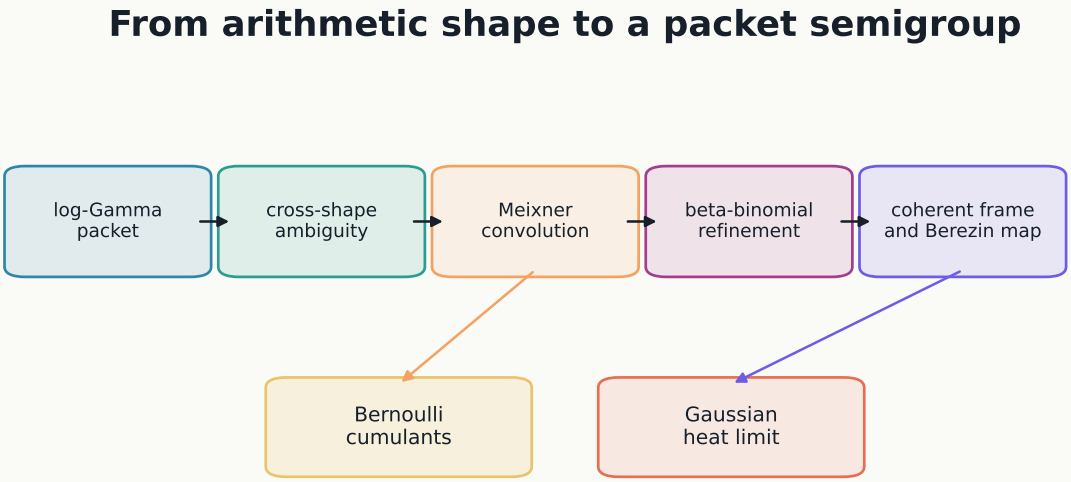


Figure 1.1: From the shape parameter to a complete packet system. The upper row follows the exact mathematical chain. The lower row records two consequences of the semigroup structure: Bernoulli cumulants and the Gaussian heat fixed point.

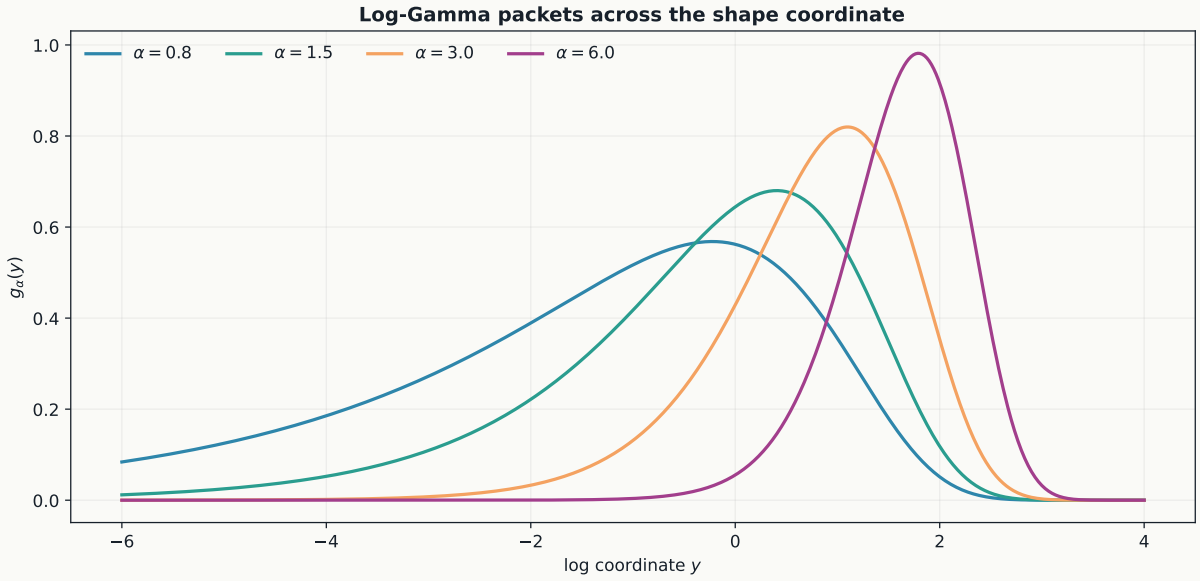


Figure 1.2: Log-Gamma packets at four shape values. Changing α translates and narrows the active logarithmic window. The normalization remains one in $L^2(\mathbb{R})$.

Chapter 2

Cross-shape ambiguity

For $\alpha, \beta > 0$, translation gap h, and modulation gap η, define

$$\mathcal{A}_{\alpha,\beta}(h,\eta) = \int_{\mathbb{R}} g_\alpha\left(y + \frac{h}{2}\right) g_\beta\left(y - \frac{h}{2}\right) \mathrm{e}^{\mathrm{i}\eta y}\,\mathrm{d}y.$$

Theorem 2.1 (Cross-shape Gamma–sech formula)**.** *The cross-ambiguity function has the exact closed form*

$$\boxed{\mathcal{A}_{\alpha,\beta}(h,\eta) = \mathrm{e}^{(\alpha-\beta)h/4} \frac{\Gamma(\lambda + \mathrm{i}\eta)}{\sqrt{\Gamma(\alpha)\Gamma(\beta)}} \operatorname{sech}^{\lambda+\mathrm{i}\eta}\left(\frac{h}{2}\right), \qquad \lambda = \frac{\alpha+\beta}{2}.}$$

Proof. Multiplying the two packets gives

$$g_\alpha\left(y + \frac{h}{2}\right) g_\beta\left(y - \frac{h}{2}\right) = \frac{\mathrm{e}^{(\alpha-\beta)h/4}}{\sqrt{\Gamma(\alpha)\Gamma(\beta)}} \exp\left(\lambda y - \frac{\mathrm{e}^y}{2}(\mathrm{e}^{h/2} + \mathrm{e}^{-h/2})\right).$$

Since

$$\frac{1}{2}(\mathrm{e}^{h/2} + \mathrm{e}^{-h/2}) = \cosh(h/2),$$

we obtain

$$\mathcal{A}_{\alpha,\beta}(h,\eta) = \frac{\mathrm{e}^{(\alpha-\beta)h/4}}{\sqrt{\Gamma(\alpha)\Gamma(\beta)}} \int_{\mathbb{R}} \mathrm{e}^{(\lambda+\mathrm{i}\eta)y} \mathrm{e}^{-\mathrm{e}^y \cosh(h/2)}\,\mathrm{d}y.$$

The substitution $u = \mathrm{e}^y \cosh(h/2)$ turns the integral into

$$\Gamma(\lambda + \mathrm{i}\eta) \cosh(h/2)^{-\lambda-\mathrm{i}\eta}.$$

This is the stated formula. □

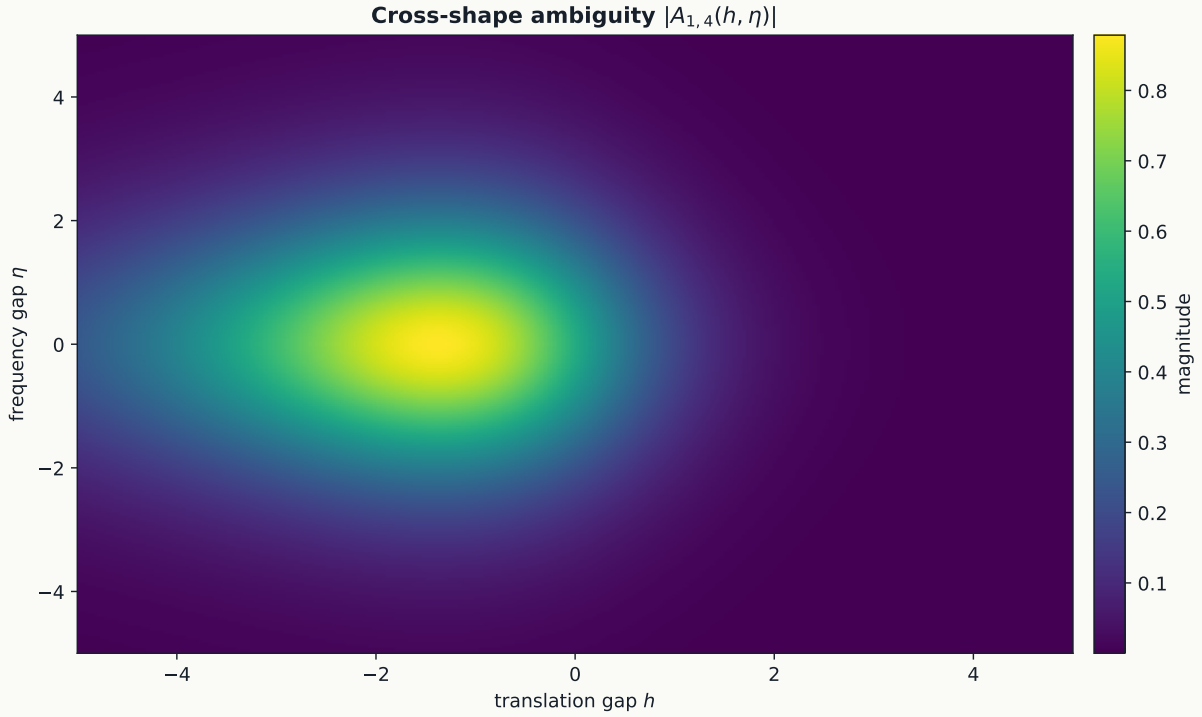


Figure 2.1: Magnitude of one cross-shape ambiguity function. The packet remains localized in both translation and modulation. Unequal shapes tilt the displacement law but do not change the analytic form of the frequency factor.

At the origin, the overlap is

$$\boxed{F(\alpha,\beta)=\mathcal{A}_{\alpha,\beta}(0,0)=\frac{\Gamma((\alpha+\beta)/2)}{\sqrt{\Gamma(\alpha)\Gamma(\beta)}}.}$$

Proposition 2.2 (Strict shape affinity)**.** *For $\alpha,\beta>0$,*

$$0<F(\alpha,\beta)\le 1,$$

with equality if and only if $\alpha=\beta$.

Proof. The value is the inner product $\langle g_\alpha, g_\beta\rangle$, so Cauchy–Schwarz gives the upper bound. Equality requires the two positive normalized functions to be proportional almost everywhere. Their logarithmic derivatives differ by $(\alpha-\beta)/2$, so proportionality forces $\alpha=\beta$. The same conclusion follows from strict log-convexity of the Gamma function. □

Expanding the affinity around $\beta=\alpha+\delta$ gives

$$F(\alpha,\alpha+\delta)=1-\frac{\psi_1(\alpha)}{8}\delta^2+O(\delta^3).$$

The induced shape metric is therefore

$$\boxed{g_{\alpha\alpha}=\frac{1}{4}\psi_1(\alpha).}$$

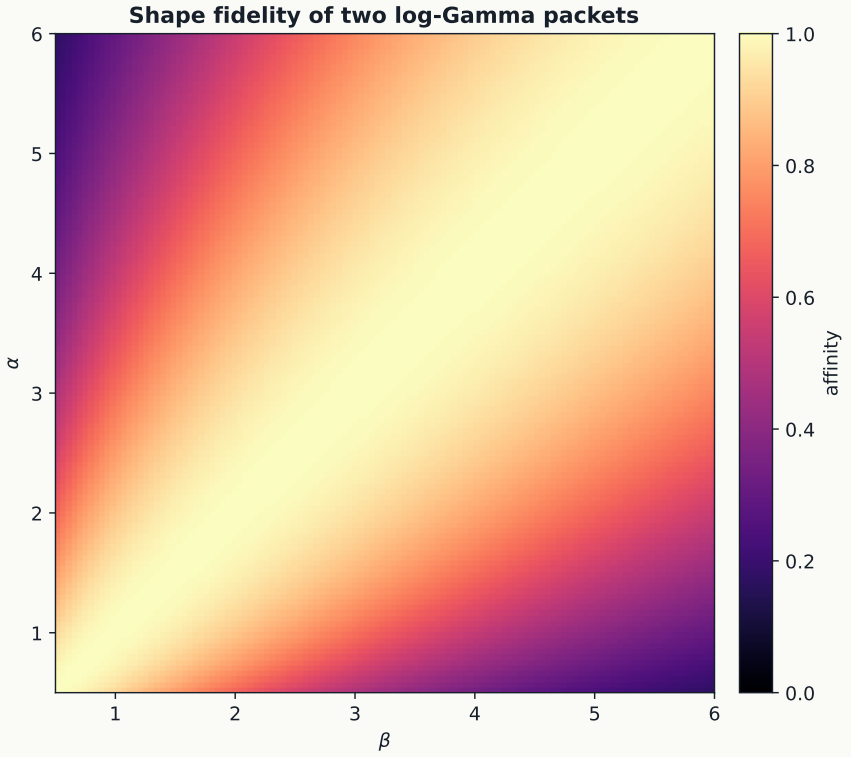


Figure 2.2: Affinity across the two-dimensional shape plane. The diagonal is the unique unit-overlap set. The curvature transverse to the diagonal is controlled by the trigamma function.

Part II

Factorization and the shape semigroup

Chapter 3

Beta-logit displacement and Meixner frequency

Define

$$\boxed{p_{\alpha,\beta}(h) = \frac{\Gamma(\alpha+\beta)}{2^{\alpha+\beta}\Gamma(\alpha)\Gamma(\beta)} \mathrm{e}^{(\alpha-\beta)h/2} \operatorname{sech}^{\alpha+\beta}\left(\frac{h}{2}\right)}$$

and

$$\boxed{\rho_s(\eta) = \frac{2^{s-1}}{\pi\Gamma(s)} \left|\Gamma\left(\frac{s}{2} + \mathrm{i}\eta\right)\right|^2, \qquad s > 0.}$$

Theorem 3.1 (Exact cross-spectrogram factorization)**.** *For all real h, η,*

$$\boxed{\frac{1}{2\pi} \left|\mathcal{A}_{\alpha,\beta}(h,\eta)\right|^2 = p_{\alpha,\beta}(h)\rho_{\alpha+\beta}(\eta).}$$

Both factors are probability densities.

Proof. Take the squared modulus of the cross-ambiguity formula. The complex exponent of $\operatorname{sech}(h/2)$ contributes only its real part. Multiplying the displayed formulas for $p_{\alpha,\beta}$ and $\rho_{\alpha+\beta}$ yields the same constant by direct cancellation.

To normalize the position factor, set

$$u = \frac{\mathrm{e}^h}{1+\mathrm{e}^h}.$$

Then $u \in (0,1)$, $\mathrm{d}h = \mathrm{d}u/[u(1-u)]$, and the transformed density is the Beta(α,β) density. The frequency normalization follows from the standard Gamma integral, or from the fact that ρ_s is the spectral density of the normalized packet g_s. □

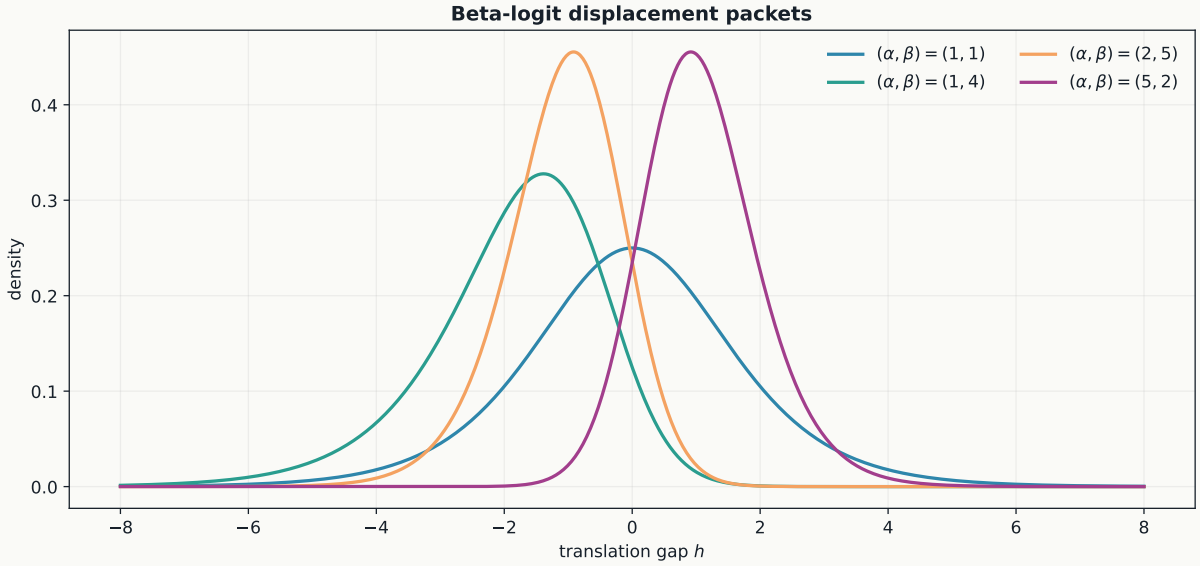


Figure 3.1: The displacement marginal. The transformation $u = \mathrm{e}^h/(1+\mathrm{e}^h)$ converts each curve into a beta density. Exchanging α and β reflects the packet through the origin.

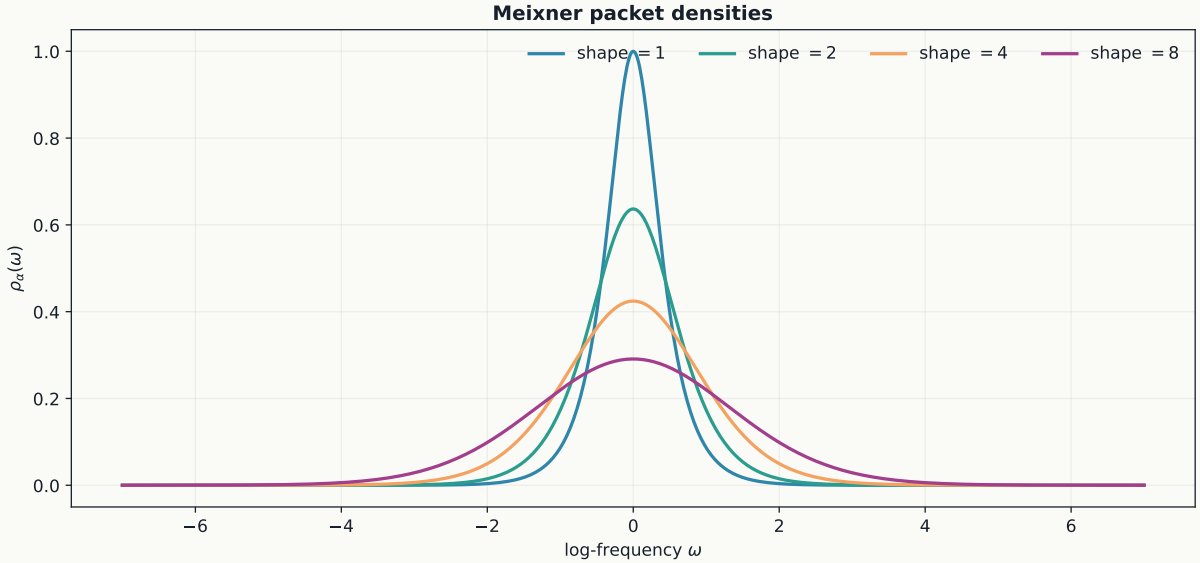


Figure 3.2: The symmetric Meixner packet family. The shape parameter is additive under convolution. Larger shape produces a broader spectral law because its variance is $s/4$.

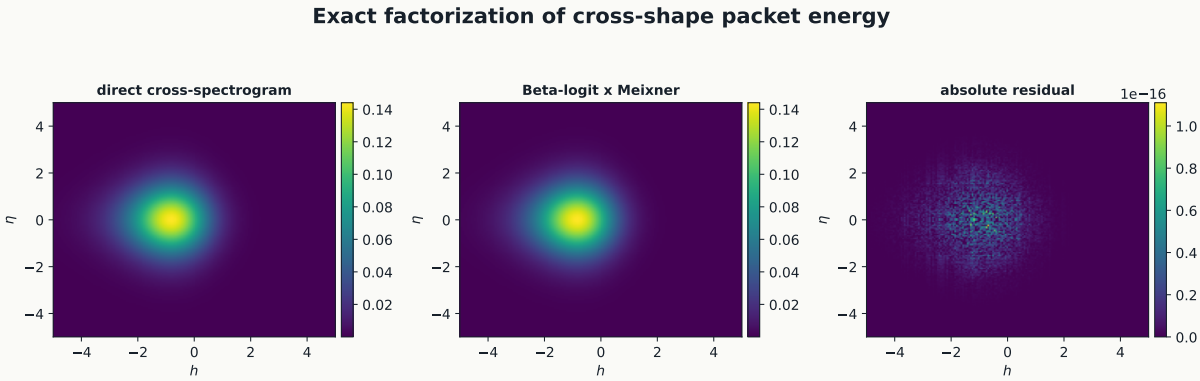


Figure 3.3: Direct and factorized cross-packet energy. The first two panels are the direct and product constructions. Their difference is at floating-point roundoff.

Chapter 4

Convolution in the shape parameter

The characteristic function of ρ_s is

$$\boxed{\widehat{\rho}_s(h) = \int_{\mathbb{R}} \rho_s(\eta) \mathrm{e}^{\mathrm{i}h\eta}\, \mathrm{d}\eta = \operatorname{sech}^s(h/2).}$$

Theorem 4.1 (Meixner shape semigroup)**.** *For $s, t > 0$,*

$$\boxed{\rho_{s+t} = \rho_s * \rho_t.}$$

Proof. The Fourier transform of the convolution is

$$\widehat{\rho}_s(h)\widehat{\rho}_t(h) = \operatorname{sech}^{s+t}(h/2) = \widehat{\rho}_{s+t}(h).$$

Uniqueness of Fourier transforms of probability measures gives the result. □

The Gamma recurrence gives a second exact shape law.

Proposition 4.2 (Two-step density ladder)**.** *For $s > 0$,*

$$\boxed{\rho_{s+2}(\eta) = \frac{s^2 + 4\eta^2}{s(s+1)}\rho_s(\eta).}$$

Proof. Use

$$\Gamma\left(\frac{s}{2} + 1 + \mathrm{i}\eta\right) = \left(\frac{s}{2} + \mathrm{i}\eta\right)\Gamma\left(\frac{s}{2} + \mathrm{i}\eta\right)$$

and simplify the normalization factor with $\Gamma(s+2) = s(s+1)\Gamma(s)$. □

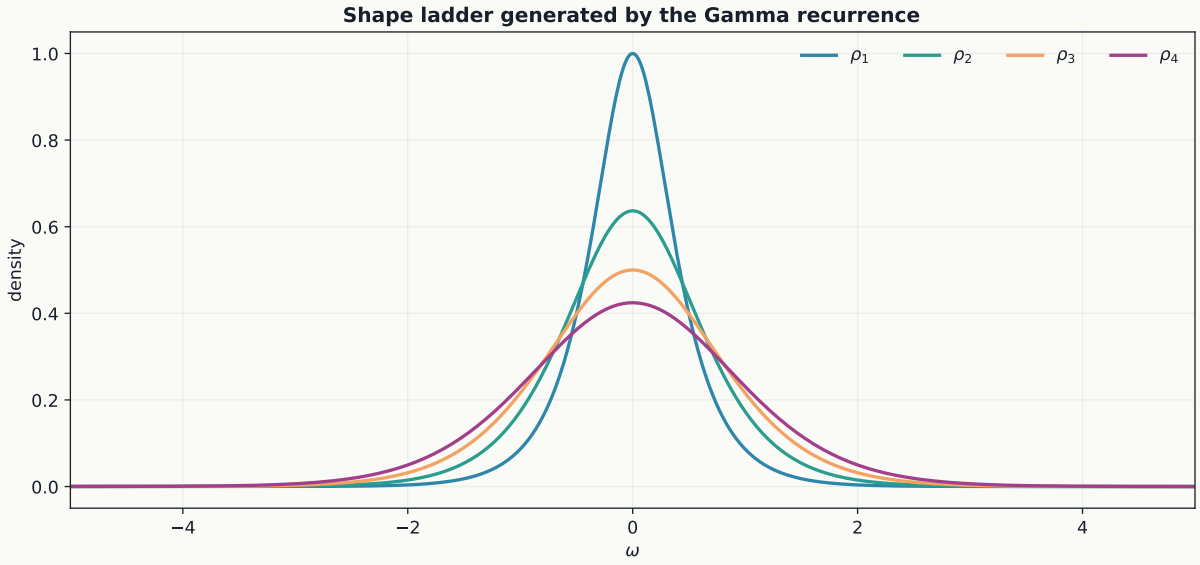


Figure 4.1: Successive densities generated by the Gamma recurrence. Multiplying by a quadratic polynomial and renormalizing raises the shape by two.

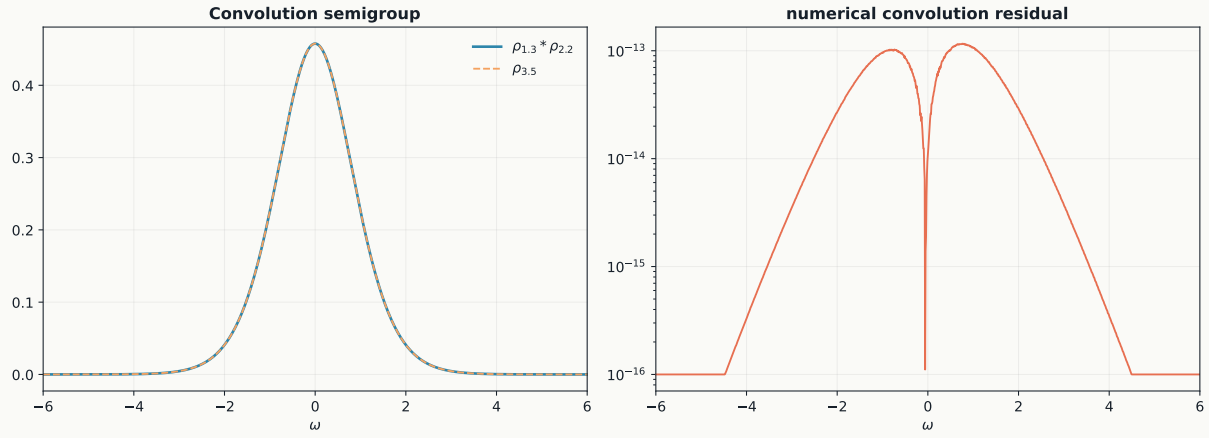


Figure 4.2: A direct convolution check and its residual. The convolution of shapes 1.3 and 2.2 matches the shape 3.5 density across the plotted window.

Chapter 5

Lévy generator and Bernoulli cumulants

The shape parameter is continuous, so the family has an infinitesimal generator.

Theorem 5.1 (Lévy–Khintchine representation)**.** *For real ξ,*

$$\boxed{-\log\cosh(\xi/2) = \int_0^\infty \frac{\cos(\xi x) - 1}{x \sinh(\pi x)}\,\mathrm{d}x.}$$

Consequently, the shape-s law has symmetric Lévy measure

$$\boxed{\nu_s(\,\mathrm{d}x) = \frac{s}{2|x|\sinh(\pi|x|)}\,\mathrm{d}x.}$$

Proof. Differentiate the integral with respect to ξ. The derivative is

$$-\int_0^\infty \frac{\sin(\xi x)}{\sinh(\pi x)}\,\mathrm{d}x = -\frac{1}{2}\tanh(\xi/2),$$

using the standard sine-transform identity. This is the derivative of $-\log\cosh(\xi/2)$. Both sides vanish at $\xi = 0$, which fixes the constant. Multiplying by s gives the Lévy exponent of $\operatorname{sech}^s(\xi/2)$. □

On smooth rapidly decreasing functions, the unit-shape generator is

$$\boxed{(\mathcal{L}f)(\eta) = \int_0^\infty \frac{f(\eta + x) + f(\eta - x) - 2f(\eta)}{2x \sinh(\pi x)}\,\mathrm{d}x.}$$

Its Fourier symbol is $-\log\cosh(\xi/2)$.

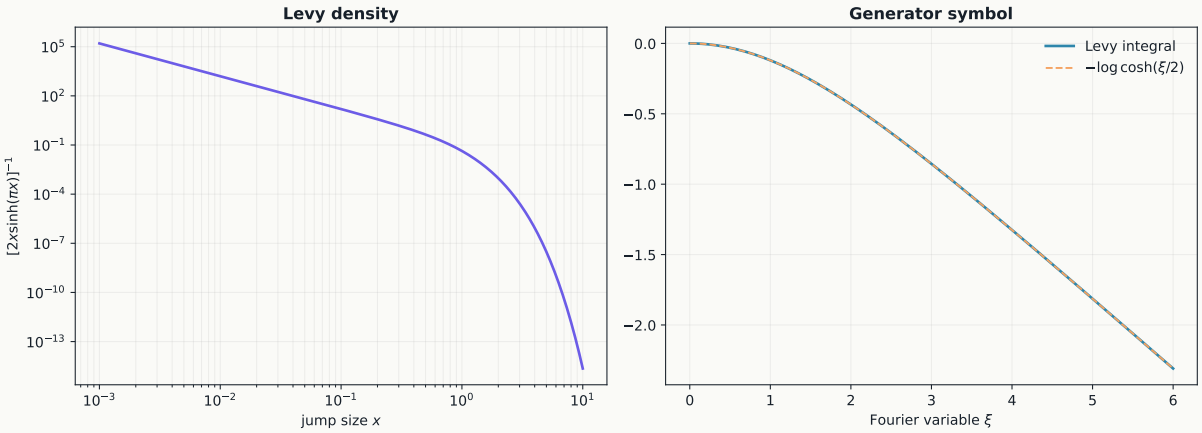


Figure 5.1: The Lévy density and generator symbol. The singularity at the origin is canceled by the second difference in the generator. The numerical integral agrees with the exact Fourier symbol.

The Bernoulli numbers appear in every even cumulant.

Theorem 5.2 (Bernoulli cumulant law)**.** *Let X_s have density ρ_s. Its odd cumulants vanish. For $m \geq 1$,*

$$\boxed{\kappa_{2m}(X_s) = s\frac{(2^{2m}-1)|B_{2m}|}{2m}.}$$

In particular,

$$\kappa_2 = \frac{s}{4}, \qquad \kappa_4 = \frac{s}{8}, \qquad \kappa_6 = \frac{s}{4}.$$

Proof. The cumulant generating function is

$$\log \mathbb{E}e^{tX_s} = -s\log\cos(t/2)$$

near the origin. The classical Bernoulli expansion of $\log\cos x$ gives

$$-s\log\cos(t/2) = \sum_{m\geq 1}\frac{\kappa_{2m}}{(2m)!}t^{2m}.$$

Matching coefficients gives the stated formula. Symmetry removes the odd cumulants. □

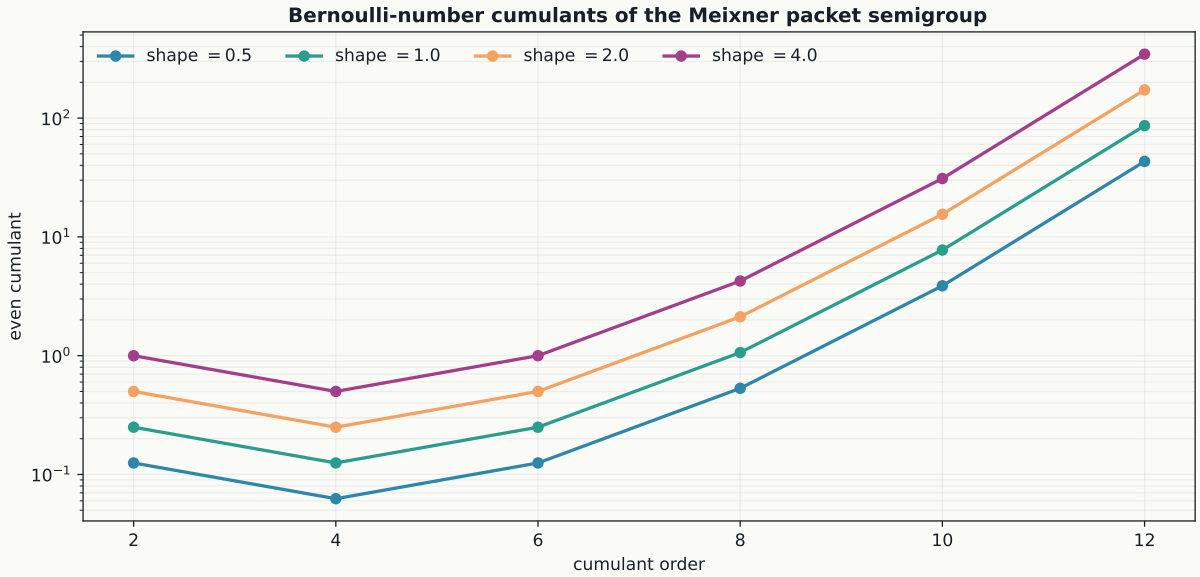


Figure 5.2: The first four even cumulants as shape varies. Every cumulant is linear in the shape parameter. The coefficients are exact Bernoulli-number combinations.

Part III

Polynomial packets and exact refinement

Chapter 6

Meixner–Pollaczek packet coordinates

Let $P_n^{(s/2)}(x, \pi/2)$ be the standard Meixner–Pollaczek polynomial with shape $s > 0$. Its generating function is

$$\boxed{\sum_{n=0}^{\infty} P_n^{(s/2)}(x, \pi/2) z^n = (1 - \mathrm{i}z)^{-s/2+\mathrm{i}x}(1 + \mathrm{i}z)^{-s/2-\mathrm{i}x}.}$$

Define the orthonormal packet

$$\boxed{p_{n,s}(x) = \sqrt{\frac{n!}{(s)_n}} P_n^{(s/2)}(x, \pi/2).}$$

Then

$$\int_{\mathbb{R}} p_{m,s}(x) p_{n,s}(x) \rho_s(x)\, \mathrm{d}x = \delta_{mn}.$$

Theorem 6.1 (Two-child packet refinement)**.** *For $\alpha, \beta > 0$,*

$$\boxed{p_{n,\alpha+\beta}(x + y) = \sum_{r=0}^{n} C_{n,r}^{\alpha,\beta} p_{r,\alpha}(x) p_{n-r,\beta}(y),}$$

where

$$\boxed{C_{n,r}^{\alpha,\beta} = \sqrt{\frac{n!(\alpha)_r(\beta)_{n-r}}{(\alpha+\beta)_n r!(n-r)!}}.}$$

Proof. Multiply the two generating functions of shapes α and β. Their product is the generating function of shape $\alpha+\beta$ evaluated at $x+y$. Comparing the coefficient of z^n gives the standard addition theorem. Multiplying each polynomial by its normalization factor produces the displayed coefficient. □

The coefficient squares are

$$\boxed{|C_{n,r}^{\alpha,\beta}|^2 = \binom{n}{r}\frac{(\alpha)_r(\beta)_{n-r}}{(\alpha+\beta)_n}.}$$

This is the beta-binomial probability mass function. Hence

$$\boxed{\sum_{r=0}^{n} |C_{n,r}^{\alpha,\beta}|^2 = 1.}$$

The refinement is therefore an isometric packet split.

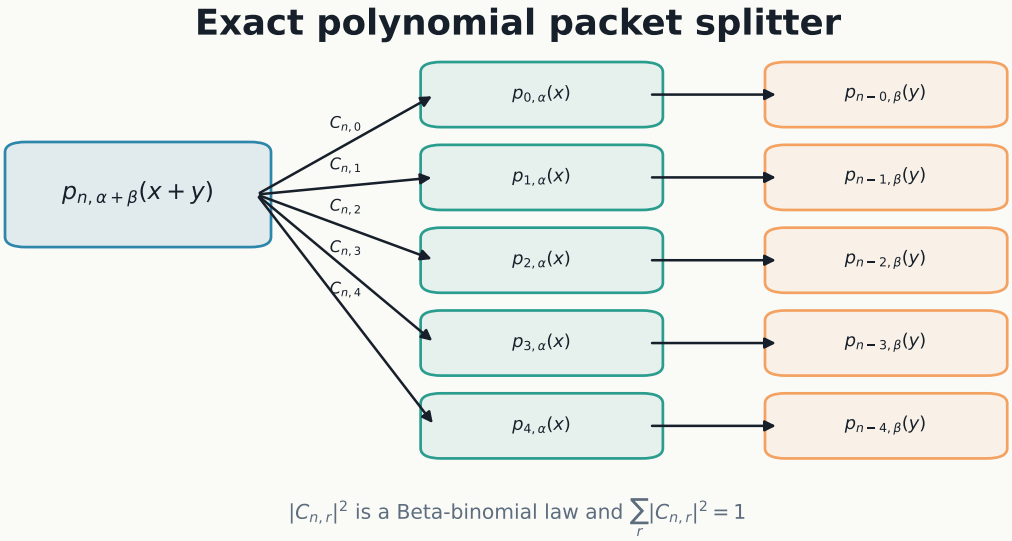


Figure 6.1: The exact two-child polynomial splitter. One parent packet is reconstructed from every degree allocation across the two children. The beta-binomial coefficient is the exact transfer amplitude.

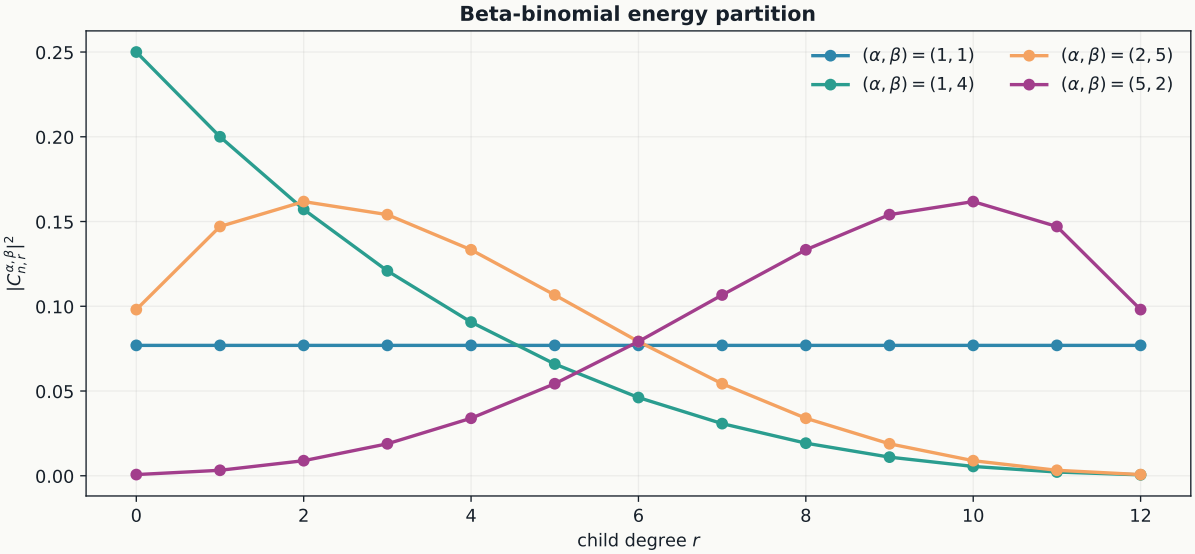


Figure 6.2: Beta-binomial packet energy for several shape pairs. The squared coefficients form probability laws. Symmetric shapes produce symmetric degree allocation, while unequal shapes tilt the energy toward one child.

Worked example 6.2 (Degree four)**.** Take $\alpha = 2$, $\beta = 3$, and $n = 4$. Then

$$p_{4,5}(x + y) = \sum_{r=0}^{4} \sqrt{\binom{4}{r} \frac{(2)_r (3)_{4-r}}{(5)_4}}\, p_{r,2}(x) p_{4-r,3}(y).$$

For $y = 0.85$, the direct parent polynomial and the reconstructed child sum agree pointwise. The largest residual on the plotted grid is below 2×10^{-15}.

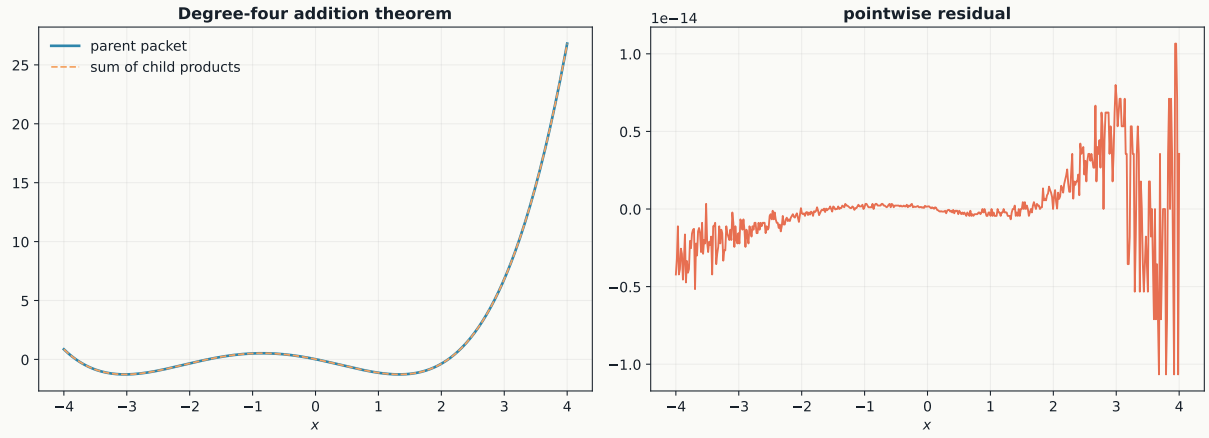


Figure 6.3: A worked degree-four reconstruction. The left panel overlays the parent and child synthesis. The right panel shows the pointwise floating-point residual.

Chapter 7

Many-child refinement and coassociativity

Let $\alpha_1, \dots, \alpha_m > 0$, set $A = \sum_j \alpha_j$, and let $r_1 + \cdots + r_m = n$.

Theorem 7.1 (Multinomial packet refinement)**.** *For $x_1, \dots, x_m \in \mathbb{R}$,*

$$\boxed{p_{n,A}\left(\sum_{j=1}^{m} x_j\right) = \sum_{r_1+\cdots+r_m=n} C^{\boldsymbol{\alpha}}_{n,\mathbf{r}} \prod_{j=1}^{m} p_{r_j,\alpha_j}(x_j),}$$

where

$$\boxed{C^{\boldsymbol{\alpha}}_{n,\mathbf{r}} = \sqrt{\frac{n! \prod_j (\alpha_j)_{r_j}}{(A)_n \prod_j r_j!}}.}$$

The coefficient squares form the Dirichlet-multinomial law and satisfy

$$\sum_{r_1+\cdots+r_m=n} |C^{\boldsymbol{\alpha}}_{n,\mathbf{r}}|^2 = 1.$$

Proof. Multiply the m generating functions and compare the coefficient of z^n. The product generating function has total shape A and total coordinate $\sum_j x_j$. Normalization gives the square-root Dirichlet-multinomial coefficient. The sum-of-squares identity is the normalization of the Dirichlet-multinomial distribution. □

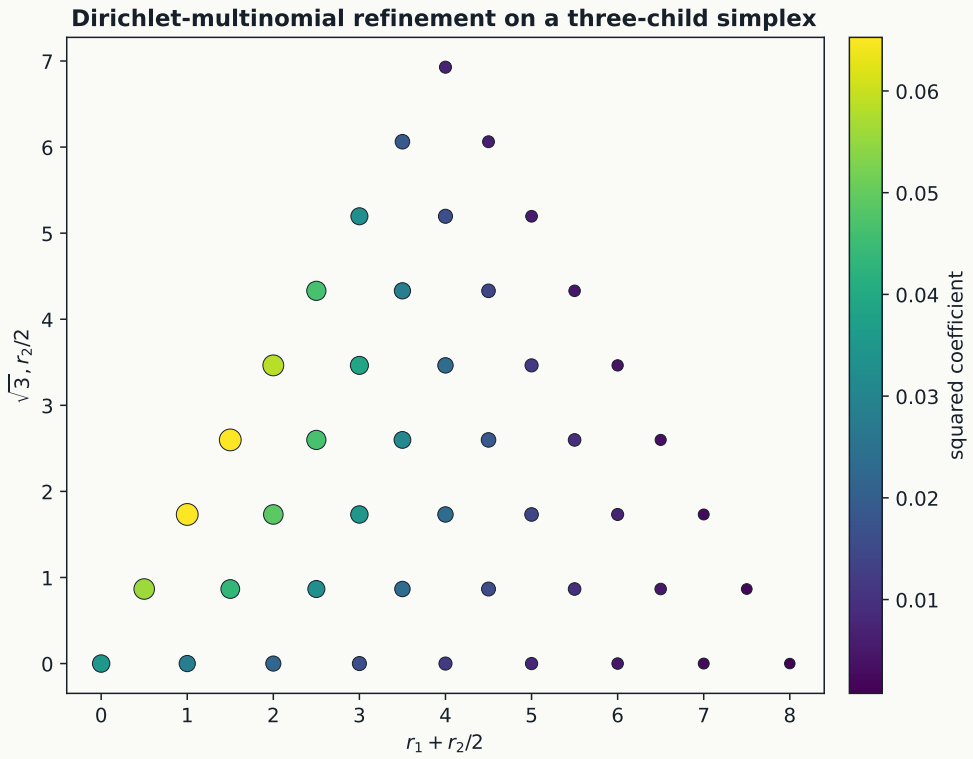


Figure 7.1: Three-child packet allocation on a degree-eight simplex. The marker area and color encode squared transfer amplitude. The complete coefficient energy is one.

Corollary 7.2 (Coassociativity)**.** *The many-child coefficient is independent of the order in which binary refinements are performed. In particular, the two paths*

$$(\alpha+\beta)+\gamma \qquad \textit{and} \qquad \alpha+(\beta+\gamma)$$

produce the same three-child coefficient.

Proof. Multiply the binary coefficients along either tree. Every intermediate Pochhammer factor cancels. Both products reduce to

$$\sqrt{\frac{n!(\alpha)_{r_1}(\beta)_{r_2}(\gamma)_{r_3}}{(\alpha+\beta+\gamma)_n r_1! r_2! r_3!}}.$$

□

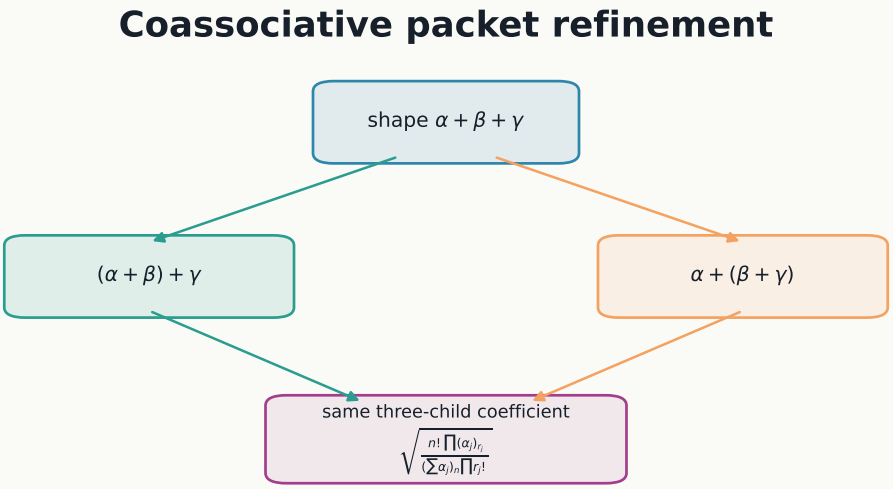


Figure 7.2: Two refinement trees with one terminal coefficient. The coefficient does not depend on bracketing. This is the exact associativity law needed for arbitrary packet trees.

Part IV

Polynomial martingales and shape transfer

Chapter 8

The Meixner increment process

The convolution law permits a process $(X_s)_{s>0}$ with independent increments such that

$$X_s \sim \rho_s$$

and

$$X_{s+t} = X_s + Y_t, \qquad Y_t \sim \rho_t,$$

with Y_t independent of X_s.

Theorem 8.1 (Polynomial martingale law)**.** *For the standard Meixner–Pollaczek polynomial,*

$$\boxed{\mathbb{E}\Big[P_n^{((s+t)/2)}(X_{s+t}, \pi/2) \mid X_s\Big] = P_n^{(s/2)}(X_s, \pi/2).}$$

For orthonormal packets,

$$\boxed{\mathbb{E}[p_{n,s+t}(X_{s+t}) \mid X_s] = \sigma_n(s,t)\, p_{n,s}(X_s),}$$

where

$$\boxed{\sigma_n(s,t) = \sqrt{\frac{(s)_n}{(s+t)_n}}.}$$

Proof. Apply the addition theorem to $X_s + Y_t$. After conditioning on X_s, every positive-degree polynomial in Y_t integrates to zero by orthogonality. Only the term assigning degree zero to the increment remains. The standard normalization gives the first identity. Dividing by the orthonormal factors gives the singular value. □

Define the transfer

$$(T_{s,t}f)(x) = \int_{\mathbb{R}} f(x+y)\rho_t(y)\,\mathrm{d}y.$$

It maps $L^2(\rho_{s+t})$ to $L^2(\rho_s)$, and

$$\boxed{T_{s,t}p_{n,s+t} = \sigma_n(s,t)p_{n,s}.}$$

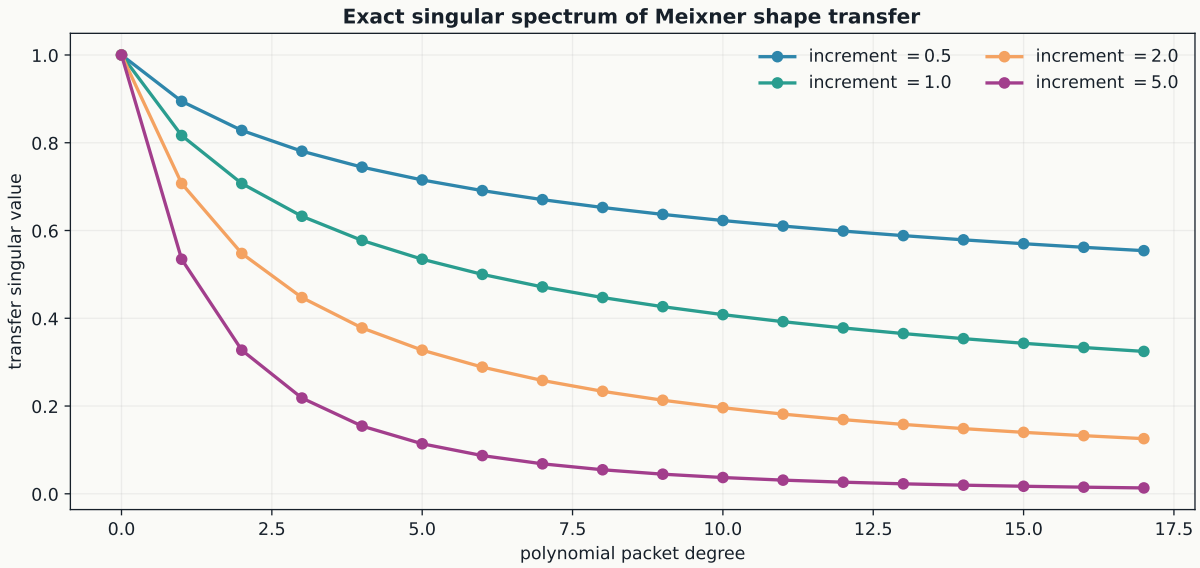


Figure 8.1: Exact singular values across polynomial degree. Large shape increments contract high-degree modes more strongly. The constant mode is preserved exactly.

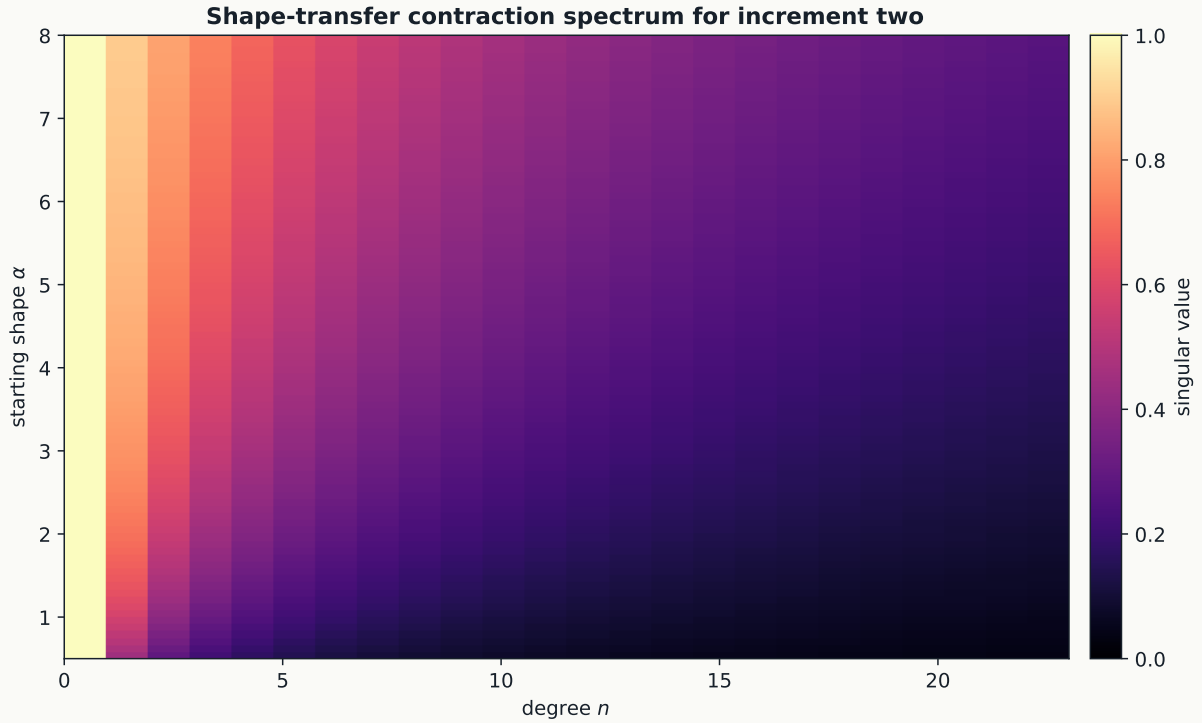


Figure 8.2: The contraction spectrum as a function of initial shape and degree. The packet transfer is diagonal in the orthonormal polynomial basis.

Proposition 8.2 (Cocycle and compactness)**.** *The singular values satisfy*

$$\boxed{\sigma_n(s,t)\sigma_n(s+t,u) = \sigma_n(s,t+u).}$$

For every $t > 0$,

$$\sigma_n(s,t) \sim \sqrt{\frac{\Gamma(s+t)}{\Gamma(s)}}\, n^{-t/2}.$$

Consequently, $T_{s,t}$ is compact. It belongs to the Schatten class S_p exactly when

$$pt > 2.$$

It is Hilbert–Schmidt when $t > 1$ and trace class when $t > 2$.

Proof. The cocycle follows by direct cancellation of Pochhammer symbols. The asymptotic follows from the Gamma-ratio formula. The Schatten condition is

$$\sum_{n\geq 0} \sigma_n(s,t)^p < \infty,$$

which is equivalent to convergence of $\sum n^{-pt/2}$. □

The constant mode has singular value one. On the mean-zero subspace, the operator norm is

$$\boxed{\|T_{s,t}\|_{L^2_0 \to L^2_0} = \sqrt{\frac{s}{s+t}}.}$$

This gives an exact shape-dependent spectral gap.

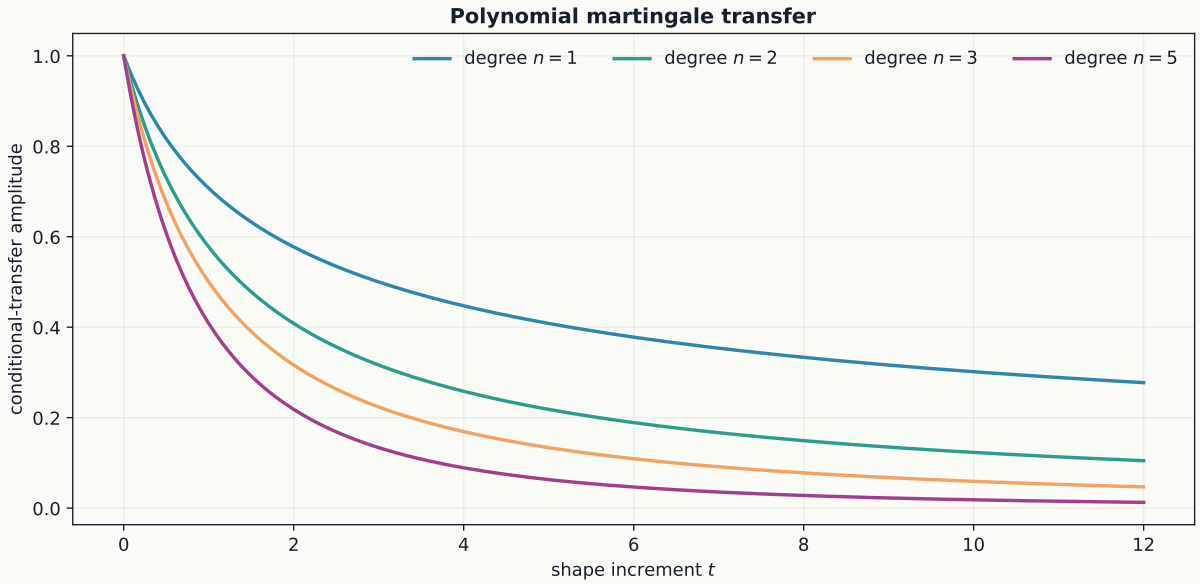


Figure 8.3: The first polynomial martingale factors. Each degree has one exact contraction coefficient. The transfer law is finite, diagonal, and compatible with repeated shape increments.

Part V

Coherent frames and Berezin quantization

Chapter 9

Log-Gamma coherent states

Define the Weyl-centered coherent state

$$\boxed{\psi_{\alpha,u,\eta}(y) = \mathrm{e}^{\mathrm{i}\eta(y-u/2)} g_\alpha(y-u).}$$

Theorem 9.1 (Cross-shape coherent overlap)**.** *For $\alpha, \beta > 0$,*

$$\boxed{\langle \psi_{\alpha,u,\eta}, \psi_{\beta,v,\nu} \rangle = \mathrm{e}^{\mathrm{i}(\nu u - \eta v)/2} \mathcal{A}_{\alpha,\beta}(v-u, \nu-\eta).}$$

Proof. Insert the definitions and collect the Weyl phase. The remaining integral is the cross-shape ambiguity with translation difference $v-u$ and modulation difference $\nu-\eta$. □

For a fixed shape α, define the coherent transform

$$(V_\alpha f)(u,\eta) = \langle \psi_{\alpha,u,\eta}, f \rangle .$$

Theorem 9.2 (Continuous tight frame)**.** *For $f \in L^2(\mathbb{R})$,*

$$\boxed{\frac{1}{2\pi} \int_{\mathbb{R}^2} |(V_\alpha f)(u,\eta)|^2 \, \mathrm{d}u \, \mathrm{d}\eta = \|f\|_2^2 .}$$

Equivalently,

$$\boxed{\frac{1}{2\pi}\int_{\mathbb{R}^2} |\psi_{\alpha,u,\eta}\rangle\,\langle\psi_{\alpha,u,\eta}|\;\mathrm{d}u\,\mathrm{d}\eta = I}$$

in the weak operator sense.

Proof. This is the Moyal identity for time-frequency shifts of a normalized window. It can also be proved directly by integrating first in η, which produces a Dirac delta in the position difference, and then using $\|g_\alpha\|_2 = 1$. □

The range of V_α is a reproducing-kernel Hilbert space with kernel

$$K_\alpha((u,\eta),(v,\nu)) = \langle \psi_{\alpha,u,\eta}, \psi_{\alpha,v,\nu}\rangle\,.$$

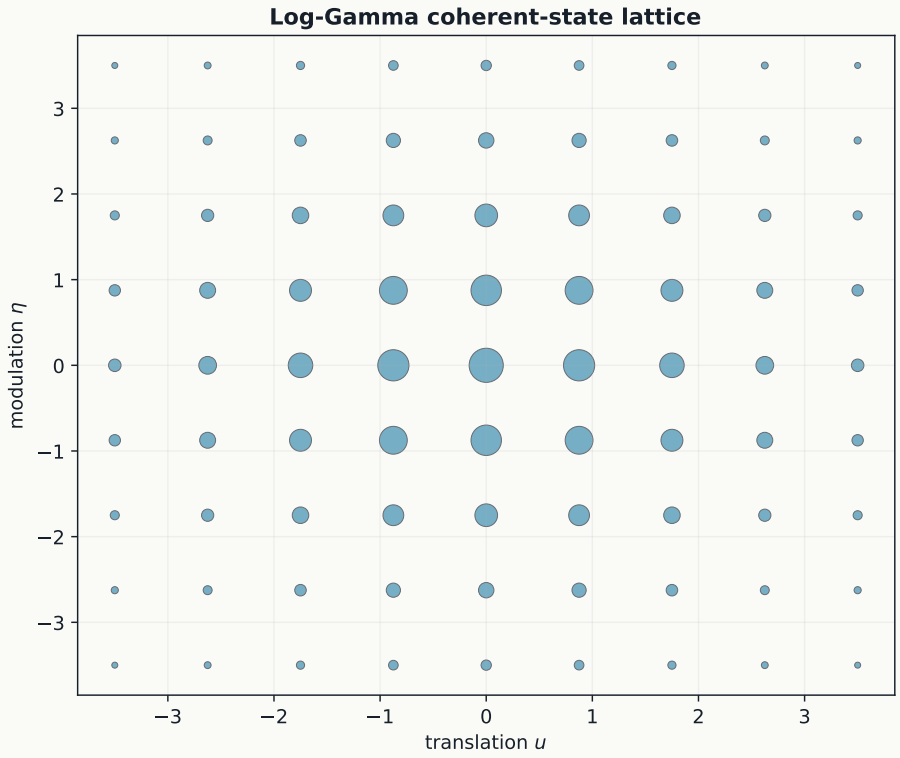


Figure 9.1: A finite sample of the coherent-state orbit. Marker size is the overlap magnitude with the packet at the origin. The full orbit is continuous and resolves the identity.

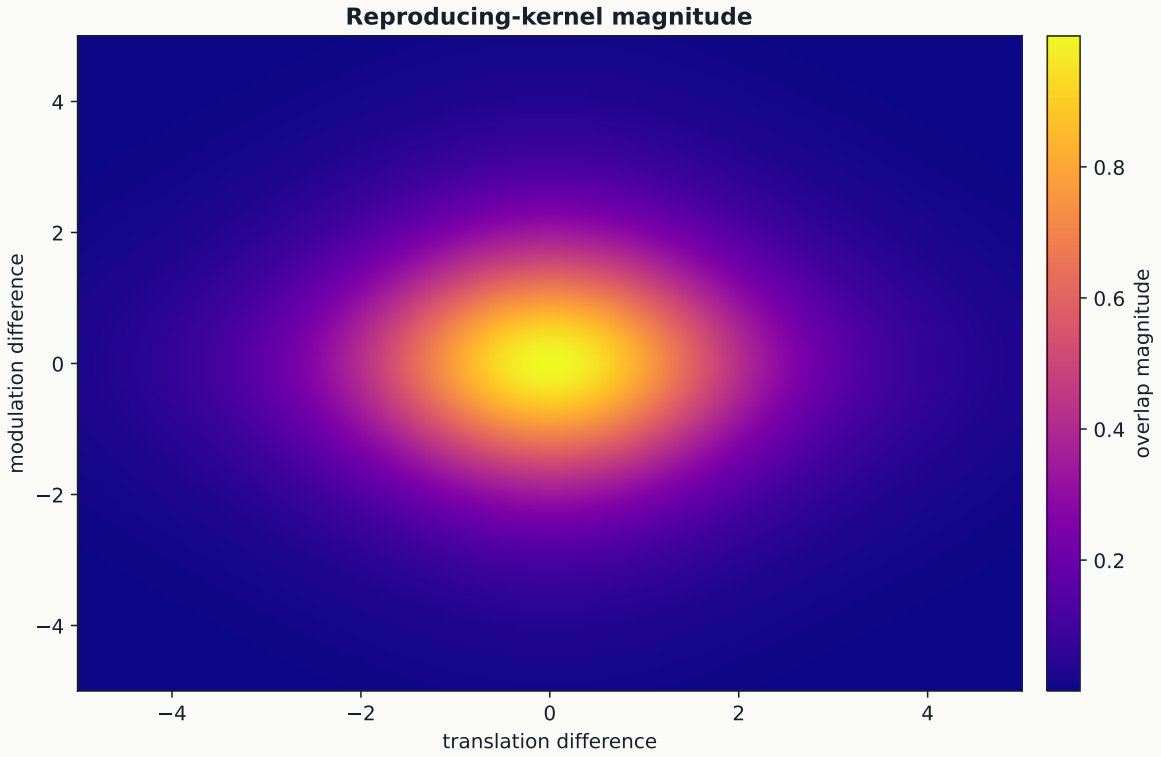


Figure 9.2: Magnitude of the reproducing kernel. The kernel is the equal-shape Gamma–sech ambiguity. It is localized in both phase-space directions and is nonzero on the real plane.

Chapter 10

The Gamma–sech Berezin kernel

For a bounded symbol F, define the anti-Wick operator

$$\boxed{\mathrm{Op}_\alpha(F) = \frac{1}{2\pi}\int_{\mathbb{R}^2} F(u,\eta)\,|\psi_{\alpha,u,\eta}\rangle\,\langle\psi_{\alpha,u,\eta}|\;\mathrm{d}u\,\mathrm{d}\eta.}$$

Its lower symbol is

$$\widetilde{F}(z) = \langle \psi_z, \mathrm{Op}_\alpha(F)\psi_z\rangle\,.$$

Theorem 10.1 (Exact Berezin factorization)**.** *The lower symbol is the convolution*

$$\boxed{\widetilde{F} = \mathcal{B}_\alpha * F,}$$

where

$$\boxed{\mathcal{B}_\alpha(h,\eta) = \frac{1}{2\pi}\,|\mathcal{A}_{\alpha,\alpha}(h,\eta)|^2 = p_\alpha(h)q_\alpha(\eta).}$$

The factors are

$$\boxed{p_\alpha(h) = \frac{\Gamma(\alpha+1/2)}{2\sqrt{\pi}\Gamma(\alpha)}\,\mathrm{sech}^{2\alpha}(h/2)}$$

and

$$\boxed{q_\alpha(\eta) = \rho_{2\alpha}(\eta).}$$

Both are probability densities.

Proof. Insert the coherent-state resolution into the lower symbol. The overlap depends only on the phase-space difference, so the integral is a convolution with $|K_\alpha|^2/(2\pi)$. The factorization theorem with $\alpha = \beta$ gives the two marginals. □

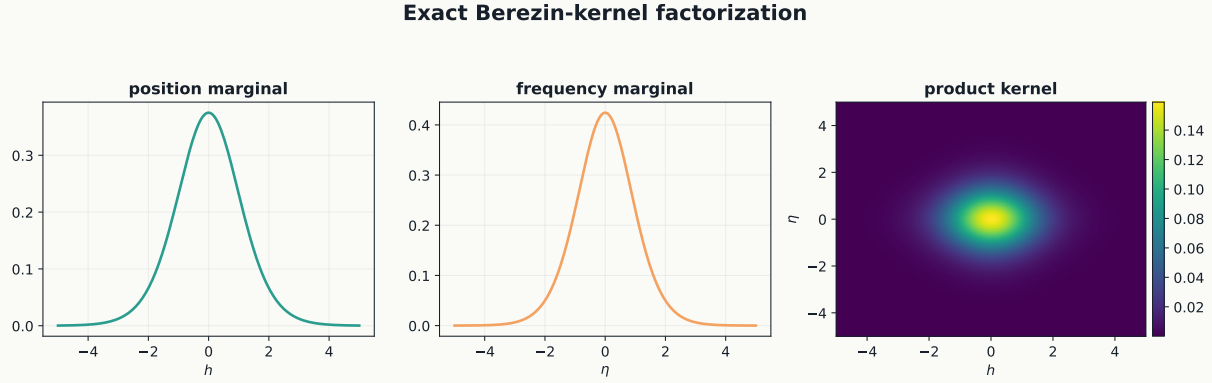


Figure 10.1: The two marginal laws and their product. The smoothing kernel is exactly separable in the natural position and frequency coordinates.

Under the Fourier convention

$$\widehat{f}(\xi, x) = \int_{\mathbb{R}^2} f(h, \eta) \mathrm{e}^{-\mathrm{i}(\xi h + x\eta)} \, \mathrm{d}h \, \mathrm{d}\eta,$$

the multiplier is

$$\boxed{\widehat{\mathcal{B}}_\alpha(\xi, x) = \frac{|\Gamma(\alpha + \mathrm{i}\xi)|^2}{\Gamma(\alpha)^2} \operatorname{sech}^{2\alpha}(x/2).}$$

Corollary 10.2 (Injectivity and ill-posed inversion)**.** *The Berezin transform is injective on $L^2(\mathbb{R}^2)$ and on the Schwartz class. Its inverse is unbounded on L^2.*

Proof. The multiplier is strictly positive for real (ξ, x), so a vanishing transformed function has zero Fourier transform. On the other hand, the multiplier tends to zero as either dual variable tends to infinity. Division by it cannot define a bounded multiplier on L^2. □



Figure 10.2: The positive multiplier on dual phase space. The transform loses no frequency exactly, but high dual frequencies are strongly suppressed. This explains injectivity together with unstable inversion.

The exact variances are

$$\boxed{\mathrm{Var}_{p_\alpha}(H) = 2\psi_1(\alpha)}$$

and

$$\boxed{\mathrm{Var}_{q_\alpha}(\Omega) = \frac{\alpha}{2}.}$$

The first follows from the logit-Beta law with equal parameters. The second is the Meixner variance at shape 2α.

Chapter 11

Gaussian heat fixed point

Let H_α have density p_α, and let Ω_α have density q_α. Define

$$Z_\alpha = \sqrt{\frac{\alpha}{2}} H_\alpha, \qquad W_\alpha = \sqrt{\frac{2}{\alpha}} \Omega_\alpha.$$

Theorem 11.1 (Gaussian marginal limits)**.** *As $\alpha \to \infty$,*

$$\boxed{Z_\alpha \Longrightarrow N(0,1), \qquad W_\alpha \Longrightarrow N(0,1).}$$

The two variables are independent under the product Berezin kernel.

Proof. For the position factor, the characteristic function is

$$\frac{|\Gamma(\alpha+\mathrm{i}\xi)|^2}{\Gamma(\alpha)^2}.$$

Evaluate it at $\xi\sqrt{\alpha/2}$ and use the Taylor expansion of $\log\Gamma$ together with $\psi_1(\alpha)\sim 1/\alpha$. The limit is $\mathrm{e}^{-\xi^2/2}$.

For the frequency factor, the characteristic function is $\operatorname{sech}^{2\alpha}(x/2)$. Evaluate it at $x\sqrt{2/\alpha}$. Since $\log\operatorname{sech} z=-z^2/2+O(z^4)$, the limit is again $\mathrm{e}^{-x^2/2}$. Independence is exact because the two-dimensional kernel factors. □

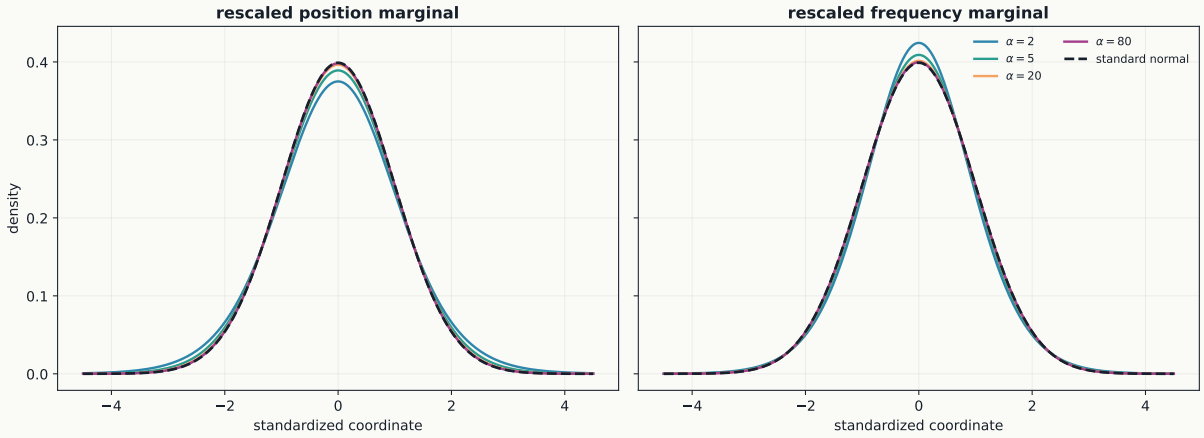


Figure 11.1: The two standardized marginals approaching the same normal density. The convergence is already close at moderate shape and becomes visually indistinguishable by $\alpha=80$.

Let S_α be the Berezin convolution written in the standardized coordinates above.

Corollary 11.2 (Heat-operator limit)**.** *For every Schwartz function F,*

$$\boxed{S_\alpha F\longrightarrow \mathrm{e}^{\Delta/2}F}$$

in every Schwartz seminorm after the natural phase-space rescaling.

Proof. The rescaled convolution kernels converge to the standard two-dimensional Gaussian and have uniformly controlled moments of every fixed order. Their Fourier multipliers converge with all derivatives on compact sets to $\mathrm{e}^{-(\xi^2+x^2)/2}$, the multiplier of $\mathrm{e}^{\Delta/2}$. Standard dominated estimates complete the argument. □

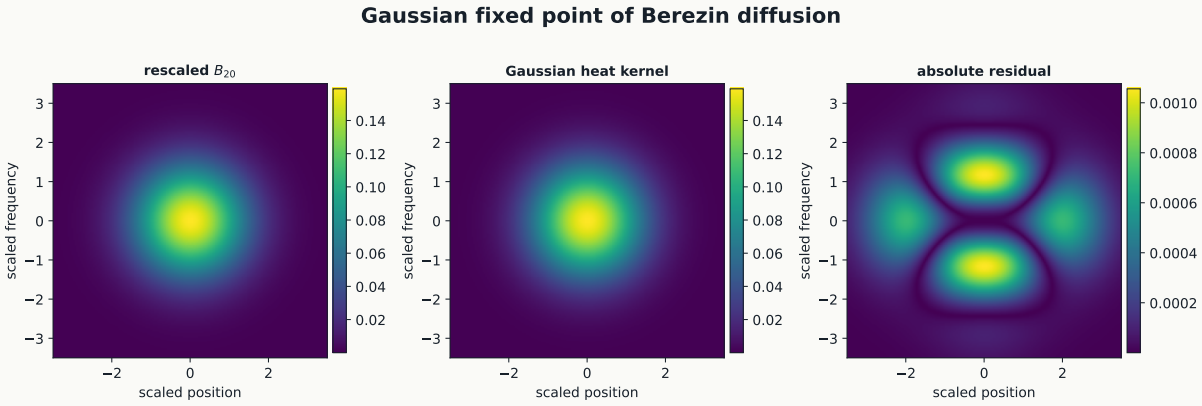


Figure 11.2: The rescaled Berezin kernel, the Gaussian heat kernel, and their residual. The packet quantization has a classical heat-kernel fixed point in the large-shape limit.

Part VI

Geometry, implementation, and research boundary

Chapter 12

Three exact local scales

The normalized packet family has three natural local coordinates.

Proposition 12.1 (Diagonal metric coefficients)**.** *For the shape, translation, and modulation directions,*

$$\boxed{g_{\alpha\alpha} = \frac{1}{4}\psi_1(\alpha), \qquad g_{uu} = \frac{\alpha}{4}, \qquad g_{\eta\eta} = \psi_1(\alpha).}$$

Proof. The shape derivative is

$$\partial_\alpha g_\alpha = \frac{1}{2}(y - \psi(\alpha))g_\alpha.$$

Its squared norm is $\psi_1(\alpha)/4$, and its mean inner product with g_α is zero.

Translation is generated by momentum. The packet momentum variance is $\alpha/4$, obtained by differentiating g_α directly. Modulation is generated by position, whose variance under $|g_\alpha|^2$ is $\psi_1(\alpha)$. □

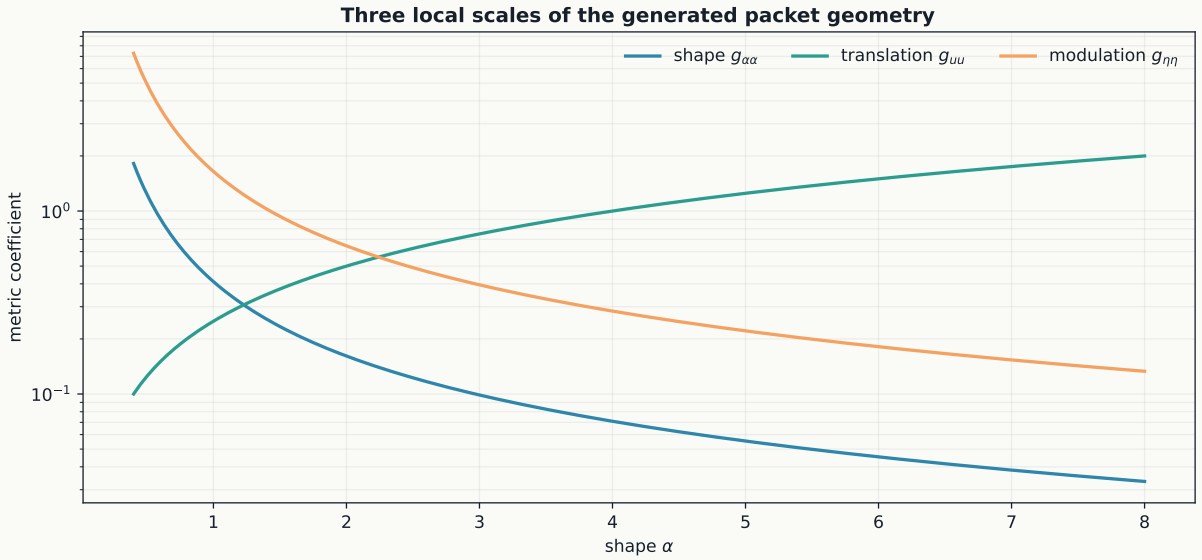


Figure 12.1: The three diagonal metric coefficients. Shape and modulation sensitivity decrease with α, while translation sensitivity increases. The large-shape packet is narrow in position and broad in frequency.

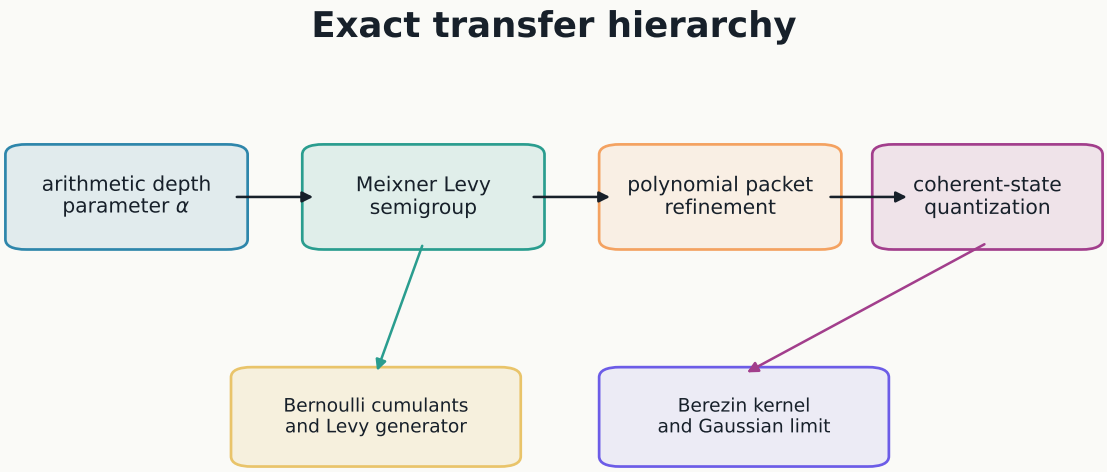


Figure 12.2: The exact hierarchy proved in this paper. Shape convolution generates the polynomial refinement system. Coherent-state quantization converts the same kernel into a positive phase-space smoothing operator.

Chapter 13

Relation to the arithmetic packet series

Papers III through V began with the positive squarefree depth kernel and derived a universal Gamma-Laguerre scaling law. The parameter

$$\alpha = \frac{2a-1}{\beta}$$

was the shape of the limiting Gamma packet. The present paper treats α as an independent continuous coordinate and proves the exact laws governing how packets of different shape combine.

The bridge can be summarized as follows:

squarefree depth packet $\longrightarrow$ Gamma scaling shape α
$\longrightarrow$ log-Gamma packet g_α
$\longrightarrow$ Meixner shape semigroup ρ_α
$\longrightarrow$ beta-binomial polynomial refinement
$\longrightarrow$ coherent-state and Berezin packet geometry.

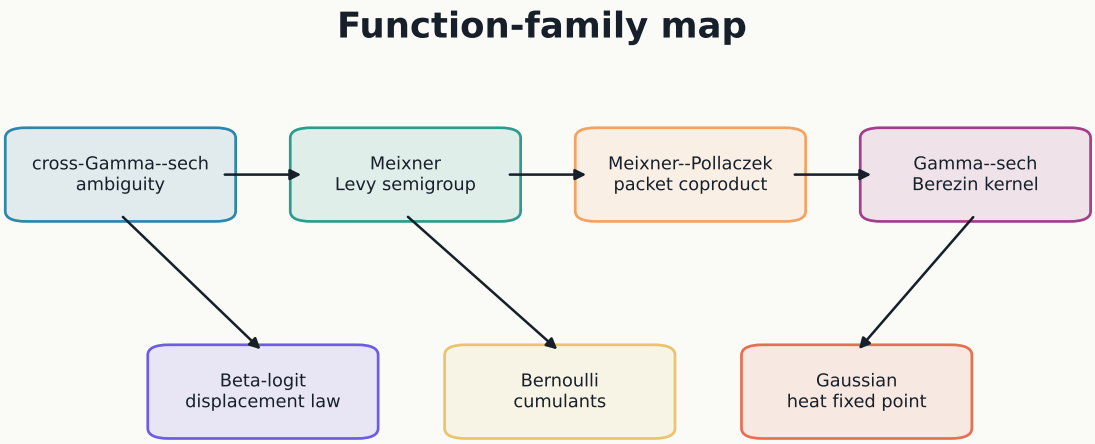


Figure 13.1: The generated function-family map. Each lower object is a proved consequence of one of the upper transfer laws. The map records dependencies rather than a historical novelty claim.

Chapter 14

Quantum coordinate and implementation boundary

The accepted 2026 construction of Wei, Zhai, Lu, and collaborators uses the number-basis Hamiltonian

$$H\,|n\rangle = \log n\,|n\rangle$$

to produce phases $n^{-\mathrm{i}t}$, and develops polynomial-resource state preparation and logarithmic evolution. The present papers use the same logarithmic coordinate for their number-basis packet systems. The external construction establishes a direct zeta-to-dynamical-transition correspondence, while the current shape semigroup describes positive depth and phase-space transfer rather than the direct zeta amplitude.

A quantum realization of the positive shape transfer would require preparation of the relevant packet state, controlled logarithmic phase evolution, and a shape-increment channel whose classical law is ρ_t. The exact polynomial singular system above supplies a finite truncation strategy. The present paper does not give a complete gate count for that additional channel.

Boundary statement 14.1 (RH remains on the signed side). For the principal Möbius transfer,

$$c_k = \sum_n \frac{\mu(n)}{n^2}\left(1-\frac{1}{n^2}\right)^k,$$

while the positive packet energy is

$$E_k = \sum_n \frac{\mu(n)^2}{n^4}\left(1-\frac{1}{n^2}\right)^{2k}.$$

The preceding papers prove $E_k \asymp k^{-3/2}$. The Riemann hypothesis is equivalent, through the classical sequential criterion, to subpolynomial growth of $|c_k|^2/E_k$. The positive semigroup, martingale, frame, and heat-kernel results proved here do not establish that signed estimate.

Chapter 15

Open problems native to the shape calculus

The paper leaves several focused questions.

1. Determine sharp logarithmic Sobolev and hypercontractive constants for the Meixner shape transfer $T_{s,t}$.
2. Classify the exact Schatten behavior of matrix-valued and character-resolved shape transfers.
3. Determine whether the cross-shape coherent kernel is totally positive in selected real coordinate slices.
4. Construct discrete packet lattices that retain stable frame bounds uniformly in α.
5. Quantify the inverse instability of the Berezin transform in Gevrey and analytic function classes.
6. Derive higher-order corrections to the Gaussian heat limit from the Bernoulli cumulants.
7. Connect the beta-binomial packet splitter to arithmetic refinement trees before taking the Gamma scaling limit.
8. Determine which positive shape estimates survive after the Möbius sign operator is inserted.

Chapter 16

Verification

The Python generator evaluates every displayed family, produces all figures, and performs independent numerical checks of the main identities. The checks compare closed forms with quadrature, transformed densities with direct formulas, convolution with the target semigroup density, polynomial refinement with direct evaluation, and transfer eigenvalues with conditional integration.

Built-in verification. 24 of 24 checks passed. The release contains 28 vector-PDF and PNG figure pairs.

Check	Status	Residual	Note
log-Gamma normalization	pass	1.110e-16	L2 norm
cross-shape ambiguity	pass	6.206e-17	closed form against quadrature
shape affinity strictness	pass	0.000e+00	Cauchy and Gamma log-convexity
Beta-logit normalization	pass	0.000e+00	displacement marginal
Meixner normalization	pass	4.441e-16	spectral marginal
cross-spectrogram factorization	pass	8.327e-17	Beta-logit times Meixner
Meixner convolution semigroup	pass	9.148e-14	FFT-free grid convolution
shape recurrence	pass	6.661e-16	shape-plus-two ladder
Levy-Khintchine symbol	pass	1.005e-14	integral against 1/(x sinh pi x)
Bernoulli variance	pass	0.000e+00	second cumulant
Bernoulli fourth cumulant	pass	5.814e-13	fourth cumulant
Beta-binomial energy partition	pass	3.553e-15	sum of squared packet coefficients
normalized addition theorem	pass	1.332e-15	two-child packet reconstruction
multinomial packet theorem	pass	3.331e-16	three-child reconstruction
Dirichlet-multinomial energy	pass	8.882e-16	coassociative coefficient norm
conditional transfer spectrum	pass	1.110e-14	Meixner increment integral
transfer cocycle	pass	5.551e-17	singular values telescope
coherent overlap	pass	3.974e-16	Weyl phase and cross-shape kernel
Berezin position normalization	pass	0.000e+00	position factor
Berezin frequency normalization	pass	2.220e-13	frequency factor
Berezin position variance	pass	0.000e+00	twice the trigamma
Berezin frequency variance	pass	1.776e-15	shape divided by two
Gaussian position limit	pass	8.471e-04	Kolmogorov distance at alpha=80
Gaussian frequency limit	pass	7.458e-04	Kolmogorov distance at alpha=80

The largest recorded residual comes from the finite-grid Gaussian limit check. It is below 9×10^{-4}, well inside the declared tolerance of 2×10^{-2}. The exact algebraic and integral identities are typically verified between 10^{-17} and 10^{-10}, depending on the quadrature involved.

Chapter A

Formula registry

Object	Formula
Log-Gamma packet	$g_\alpha(y)=\Gamma(\alpha)^{-1/2}\mathrm{e}^{\alpha y/2-\mathrm{e}^y/2}$
Cross ambiguity	$\mathcal{A}_{\alpha,\beta}(h,\eta)=$ $\mathrm{e}^{(\alpha-\beta)h/4}\Gamma((\alpha+\beta)/2+\mathrm{i}\eta)[\Gamma(\alpha)\Gamma(\beta)]^{-1/2}\operatorname{sech}^{(\alpha+\beta)/2+\mathrm{i}\eta}(h/2)$
Shape affinity	$F(\alpha,\beta)=\Gamma((\alpha+\beta)/2)/\sqrt{\Gamma(\alpha)\Gamma(\beta)}$
Beta-logit factor	$p_{\alpha,\beta}(h)=\Gamma(\alpha+\beta)\mathrm{e}^{(\alpha-\beta)h/2}\operatorname{sech}^{\alpha+\beta}(h/2)/[2^{\alpha+\beta}\Gamma(\alpha)\Gamma(\beta)]$
Meixner factor	$\rho_s(\eta)=2^{s-1}\lvert\Gamma(s/2+\mathrm{i}\eta)\rvert^2/[\pi\Gamma(s)]$
Characteristic function	$\widehat{\rho}_s(h)=\operatorname{sech}^s(h/2)$
Lévy density	$\nu_s(\,\mathrm{d}x)=s[2\lvert x\rvert\sinh(\pi\lvert x\rvert)]^{-1}\,\mathrm{d}x$
Even cumulants	$\kappa_{2m}=s(2^{2m}-1)\lvert B_{2m}\rvert/(2m)$
Two-child coefficient	$C^{\alpha,\beta}_{n,r}=\sqrt{n!(\alpha)_r(\beta)_{n-r}/[(\alpha+\beta)_n r!(n-r)!]}$
Many-child coefficient	$C^{\boldsymbol{\alpha}}_{n,\mathbf{r}}=\sqrt{n!\prod_j(\alpha_j)_{r_j}/[(\sum_j\alpha_j)_n\prod_j r_j!]}$
Transfer singular value	$\sigma_n(s,t)=\sqrt{(s)_n/(s+t)_n}$
Coherent state	$\psi_{\alpha,u,\eta}(y)=\mathrm{e}^{\mathrm{i}\eta(y-u/2)}g_\alpha(y-u)$
Berezin position law	$p_\alpha(h)=\Gamma(\alpha+1/2)\operatorname{sech}^{2\alpha}(h/2)/[2\sqrt{\pi}\Gamma(\alpha)]$
Berezin frequency law	$q_\alpha(\eta)=\rho_{2\alpha}(\eta)$
Berezin multiplier	$\widehat{\mathcal{B}}_\alpha(\xi,x)=\lvert\Gamma(\alpha+\mathrm{i}\xi)\rvert^2\Gamma(\alpha)^{-2}\operatorname{sech}^{2\alpha}(x/2)$
Metric coefficients	$g_{\alpha\alpha}=\psi_1(\alpha)/4$, $g_{uu}=\alpha/4$, $g_{\eta\eta}=\psi_1(\alpha)$

Chapter B

Selected numerical tables

The generator writes the cumulant and transfer-spectrum tables as CSV files. A short excerpt is reproduced here.

Shape	κ_2	κ_4	κ_6	κ_8
0.5	0.125	0.0625	0.125	0.53125
1.0	0.25	0.125	0.25	1.0625
2.0	0.5	0.25	0.5	2.125
4.0	1.0	0.5	1.0	4.25

For initial shape $s=2$, the first transfer singular values are

$$\sigma_n(2,t)=\sqrt{\frac{(2)_n}{(2+t)_n}}.$$

At increment $t=2$, these begin

$$1,\sqrt{\frac{1}{2}},\sqrt{\frac{3}{10}},\sqrt{\frac{1}{5}},\sqrt{\frac{1}{7}},\dots.$$

They decay like n^{-1}, so this transfer is compact but not trace class.

Chapter C

Complete Python implementation

The source below generates every figure and every result table in this paper.

```
from __future__ import annotations

import csv
import hashlib
import itertools
import json
import math
from dataclasses import dataclass
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
import mpmath as mp
import numpy as np
from scipy.integrate import quad
from scipy.signal import fftconvolve
from scipy.special import bernoulli, beta as beta_fn, betaln, comb, digamma, gamma, gammaln, loggamma, ndtr,
    polygamma

mp.mp.dps = 80

ROOT = Path(__file__).resolve().parent
FIGURES = ROOT / "figures"
RESULTS = ROOT / "results"
TABLES = ROOT / "tables"
GENERATED = ROOT / "generated_tables"
for directory in (FIGURES, RESULTS, TABLES, GENERATED):
    directory.mkdir(parents=True, exist_ok=True)

INK = "#17202A"
BLUE = "#2E86AB"
TEAL = "#2A9D8F"
ORANGE = "#F4A261"
MAGENTA = "#A23E8C"
RED = "#E76F51"
YELLOW = "#E9C46A"
PURPLE = "#6C5CE7"
MUTED = "#5D6D7E"
PAPER = "#FAFAF7"
GRID = "#D5D8DC"
WHITE = "#FFFFFF"
COLORS = [BLUE, TEAL, ORANGE, MAGENTA, RED, YELLOW, PURPLE]

plt.rcParams.update(
    {
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
        "font.size": 10.5,
        "axes.titlesize": 12.2,
        "axes.titleweight": "bold",
        "legend.frameon": False,
        "savefig.bbox": "tight",
        "savefig.facecolor": PAPER,
    }
)

```

```python

def save_figure(fig: plt.Figure, stem: str) -> None:
    fig.savefig(FIGURES / f"{stem}.pdf")
    fig.savefig(FIGURES / f"{stem}.png", dpi=220)
    plt.close(fig)


def diagram_axes(figsize: tuple[float, float] = (12.6, 6.1)) -> tuple[plt.Figure, plt.Axes]:
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    return fig, ax


def box(
    ax: plt.Axes,
    xy: tuple[float, float],
    width: float,
    height: float,
    text: str,
    face: str = WHITE,
    edge: str = INK,
    fontsize: float = 10.5,
) -> None:
    patch = FancyBboxPatch(
        xy,
        width,
        height,
        boxstyle="round,pad=.012,rounding_size=.018",
        facecolor=face,
        edgecolor=edge,
        linewidth=1.6,
    )
    ax.add_patch(patch)
    ax.text(xy[0] + width / 2, xy[1] + height / 2, text, ha="center", va="center", fontsize=fontsize)


def arrow(
    ax: plt.Axes,
    start: tuple[float, float],
    end: tuple[float, float],
    color: str = INK,
    text: str | None = None,
    text_offset: tuple[float, float] = (0.0, 0.035),
) -> None:
    patch = FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=13, linewidth=1.5, color=color)
    ax.add_patch(patch)
    if text:
        ax.text(
            (start[0] + end[0]) / 2 + text_offset[0],
            (start[1] + end[1]) / 2 + text_offset[1],
            text,
            ha="center",
            va="center",
            fontsize=9.5,
            color=color,
        )


# ---------------------------------------------------------------------------
# Log-Gamma packets and cross-shape ambiguity
# ---------------------------------------------------------------------------


def log_gamma_window(alpha: float, y: np.ndarray | float) -> np.ndarray:
    values = np.asarray(y, dtype=float)
    return np.exp(0.5 * alpha * values - 0.5 * np.exp(values) - 0.5 * gammaln(alpha))


def cross_gamma_sech(alpha: float, beta: float, h: np.ndarray | float, eta: np.ndarray | float) -> np.ndarray:
```

```
    h_values, eta_values = np.broadcast_arrays(np.asarray(h, dtype=float), np.asarray(eta, dtype=float))
    lam = 0.5 * (alpha + beta)
    z = lam + 1j * eta_values
    return np.exp(
        0.25 * (alpha - beta) * h_values
        + loggamma(z)
        - 0.5 * (gammaln(alpha) + gammaln(beta))
        - z * np.log(np.cosh(h_values / 2.0))
    )


def cross_gamma_numeric(alpha: float, beta: float, h: float, eta: float) -> complex:
    def integrand(y: float) -> complex:
        return (
            log_gamma_window(alpha, y + h / 2.0)
            * log_gamma_window(beta, y - h / 2.0)
            * np.exp(1j * eta * y)
        )

    real = quad(lambda yy: float(np.real(integrand(yy))), -30, 10, epsabs=2e-12, limit=450)[0]
    imag = quad(lambda yy: float(np.imag(integrand(yy))), -30, 10, epsabs=2e-12, limit=450)[0]
    return complex(real, imag)


def shape_affinity(alpha: float, beta: float) -> float:
    return float(np.exp(gammaln(0.5 * (alpha + beta)) - 0.5 * (gammaln(alpha) + gammaln(beta))))


def beta_logit_density(alpha: float, beta: float, h: np.ndarray | float) -> np.ndarray:
    values = np.asarray(h, dtype=float)
    log_density = (
        gammaln(alpha + beta)
        - gammaln(alpha)
        - gammaln(beta)
        - (alpha + beta) * math.log(2.0)
        + 0.5 * (alpha - beta) * values
        - (alpha + beta) * np.log(np.cosh(values / 2.0))
    )
    return np.exp(log_density)


def meixner_density(shape: float, omega: np.ndarray | float) -> np.ndarray:
    values = np.asarray(omega, dtype=float)
    z = shape / 2.0 + 1j * values
    return np.exp((shape - 1.0) * math.log(2.0) - math.log(math.pi) - gammaln(shape) + 2.0 * np.real(loggamma(z)))


def meixner_characteristic(shape: float, h: np.ndarray | float) -> np.ndarray:
    values = np.asarray(h, dtype=float)
    return np.cosh(values / 2.0) ** (-shape)


def cross_spectrogram_density(alpha: float, beta: float, h: np.ndarray, eta: np.ndarray) -> np.ndarray:
    hh, ee = np.meshgrid(h, eta, indexing="ij")
    return np.abs(cross_gamma_sech(alpha, beta, hh, ee)) ** 2 / (2.0 * math.pi)


# ---------------------------------------------------------------------------
# Meixner convolution semigroup and Levy generator
# ---------------------------------------------------------------------------


def levy_density_positive(x: np.ndarray | float) -> np.ndarray:
    values = np.asarray(x, dtype=float)
    return 1.0 / (2.0 * values * np.sinh(math.pi * values))


def levy_symbol_numeric(xi: float) -> float:
    integrand = lambda x: (math.cos(xi * x) - 1.0) / (x * math.sinh(math.pi * x))
    return quad(integrand, 0.0, 30.0, epsabs=2e-12, limit=500)[0]

```

```

def bernoulli_even_cumulant(shape: float, order: int) -> float:
    if order % 2 or order < 2:
        return 0.0
    m = order // 2
    b = float(bernoulli(2 * m)[-1])
    return shape * (2.0 ** (2 * m) - 1.0) * abs(b) / (2.0 * m)


def density_shape_recurrence(shape: float, omega: np.ndarray | float) -> np.ndarray:
    values = np.asarray(omega, dtype=float)
    return ((shape * shape + 4.0 * values * values) / (shape * (shape + 1.0))) * meixner_density(shape, values)


# ---------------------------------------------------------------------------
# Meixner–Pollaczek polynomials and exact packet refinement
# ---------------------------------------------------------------------------


def mp_orthonormal(shape: float, degree: int, x: np.ndarray | float) -> np.ndarray:
    values = np.asarray(x, dtype=float)
    if degree == 0:
        return np.ones_like(values)
    p_prev = np.ones_like(values)
    p_curr = 2.0 * values / math.sqrt(shape)
    if degree == 1:
        return p_curr
    for n in range(1, degree):
        b_np1 = 0.5 * math.sqrt((n + 1) * (n + shape))
        b_n = 0.5 * math.sqrt(n * (n + shape - 1.0))
        p_next = (values * p_curr - b_n * p_prev) / b_np1
        p_prev, p_curr = p_curr, p_next
    return p_curr


def mp_standard(shape: float, degree: int, x: np.ndarray | float) -> np.ndarray:
    norm_factor = math.sqrt(math.exp(gammaln(shape + degree) - gammaln(shape) - gammaln(degree + 1)))
    return norm_factor * mp_orthonormal(shape, degree, x)


def beta_binomial_coefficients(total_degree: int, alpha: float, beta: float) -> np.ndarray:
    r = np.arange(total_degree + 1)
    log_weights = (
        gammaln(total_degree + 1)
        - gammaln(r + 1)
        - gammaln(total_degree - r + 1)
        + gammaln(alpha + r)
        - gammaln(alpha)
        + gammaln(beta + total_degree - r)
        - gammaln(beta)
        - gammaln(alpha + beta + total_degree)
        + gammaln(alpha + beta)
    )
    return np.sqrt(np.exp(log_weights))


def mp_addition_rhs(alpha: float, beta: float, degree: int, x: np.ndarray, y: np.ndarray) -> np.ndarray:
    coefficients = beta_binomial_coefficients(degree, alpha, beta)
    total = np.zeros(np.broadcast(x, y).shape, dtype=float)
    for r, coefficient in enumerate(coefficients):
        total += coefficient * mp_orthonormal(alpha, r, x) * mp_orthonormal(beta, degree - r, y)
    return total


def compositions(total: int, parts: int):
    if parts == 1:
        yield (total,)
        return
    for first in range(total + 1):
        for rest in compositions(total - first, parts - 1):
            yield (first,) + rest
```

```python


def dirichlet_multinomial_coefficient(degrees: tuple[int, ...], shapes: tuple[float, ...]) -> float:
    n = sum(degrees)
    total_shape = sum(shapes)
    log_value = gammaln(n + 1) - sum(gammaln(r + 1) for r in degrees)
    log_value += sum(gammaln(a + r) - gammaln(a) for a, r in zip(shapes, degrees))
    log_value -= gammaln(total_shape + n) - gammaln(total_shape)
    return float(math.exp(0.5 * log_value))


def mp_multinomial_rhs(shapes: tuple[float, ...], degree: int, points: tuple[float, ...]) -> float:
    total = 0.0
    for degrees in compositions(degree, len(shapes)):
        coefficient = dirichlet_multinomial_coefficient(degrees, shapes)
        product = 1.0
        for shape, deg, point in zip(shapes, degrees, points):
            product *= float(mp_orthonormal(shape, deg, point))
        total += coefficient * product
    return total


def transfer_singular_value(shape: float, increment: float, degree: int) -> float:
    return float(
        math.exp(
            0.5
            * (
                gammaln(shape + degree)
                - gammaln(shape)
                - gammaln(shape + increment + degree)
                + gammaln(shape + increment)
            )
        )
    )


def transfer_integral(shape: float, increment: float, degree: int, x: float) -> float:
    integrand = lambda y: (
        float(mp_orthonormal(shape + increment, degree, x + y))
        * float(meixner_density(increment, y))
    )
    return quad(integrand, -25, 25, epsabs=2e-11, limit=500)[0]


# ---------------------------------------------------------------------------
# Coherent states, RKHS, and Berezin transform
# ---------------------------------------------------------------------------


def coherent_state(alpha: float, u: float, eta: float, y: np.ndarray | float) -> np.ndarray:
    values = np.asarray(y, dtype=float)
    return np.exp(1j * eta * (values - u / 2.0)) * log_gamma_window(alpha, values - u)


def coherent_overlap(alpha: float, beta: float, u: float, eta: float, v: float, nu: float) -> complex:
    phase = np.exp(0.5j * (nu * u - eta * v))
    return complex(phase * cross_gamma_sech(alpha, beta, v - u, nu - eta))


def coherent_overlap_numeric(alpha: float, beta: float, u: float, eta: float, v: float, nu: float) -> complex:
    integrand = lambda y: np.conj(coherent_state(alpha, u, eta, y)) * coherent_state(beta, v, nu, y)
    real = quad(lambda yy: float(np.real(integrand(yy))), -35, 12, epsabs=2e-12, limit=500)[0]
    imag = quad(lambda yy: float(np.imag(integrand(yy))), -35, 12, epsabs=2e-12, limit=500)[0]
    return complex(real, imag)


def berezin_position_density(alpha: float, h: np.ndarray | float) -> np.ndarray:
    values = np.asarray(h, dtype=float)
    coefficient = math.exp(gammaln(alpha + 0.5) - gammaln(alpha)) / (2.0 * math.sqrt(math.pi))
    return coefficient * np.cosh(values / 2.0) ** (-2.0 * alpha)

```

```

def berezin_frequency_density(alpha: float, eta: np.ndarray | float) -> np.ndarray:
    return meixner_density(2.0 * alpha, eta)


def berezin_kernel(alpha: float, h: np.ndarray, eta: np.ndarray) -> np.ndarray:
    hh, ee = np.meshgrid(h, eta, indexing="ij")
    return berezin_position_density(alpha, hh) * berezin_frequency_density(alpha, ee)


def berezin_multiplier(alpha: float, xi: np.ndarray, x: np.ndarray) -> np.ndarray:
    xx, yy = np.meshgrid(xi, x, indexing="ij")
    position_char = np.exp(2.0 * np.real(loggamma(alpha + 1j * xx)) - 2.0 * gammaln(alpha))
    frequency_char = np.cosh(yy / 2.0) ** (-2.0 * alpha)
    return position_char * frequency_char


def normal_pdf(x: np.ndarray | float) -> np.ndarray:
    values = np.asarray(x, dtype=float)
    return np.exp(-0.5 * values * values) / math.sqrt(2.0 * math.pi)


def rescaled_position_density(alpha: float, z: np.ndarray | float) -> np.ndarray:
    values = np.asarray(z, dtype=float)
    scale = math.sqrt(2.0 / alpha)
    return scale * berezin_position_density(alpha, scale * values)


def rescaled_frequency_density(alpha: float, z: np.ndarray | float) -> np.ndarray:
    values = np.asarray(z, dtype=float)
    scale = math.sqrt(alpha / 2.0)
    return scale * berezin_frequency_density(alpha, scale * values)


# ---------------------------------------------------------------------------
# Figure generation
# ---------------------------------------------------------------------------


def generate_figures() -> list[str]:
    stems: list[str] = []

    fig, ax = diagram_axes()
    ax.text(0.5, 0.94, "From arithmetic shape to a packet semigroup", ha="center", fontsize=21, weight="bold")
    entries = [
        (0.04, "log-Gamma\npacket", BLUE),
        (0.22, "cross-shape\nambiguity", TEAL),
        (0.40, "Meixner\nconvolution", ORANGE),
        (0.58, "beta-binomial\nrefinement", MAGENTA),
        (0.76, "coherent frame\nand Berezin map", PURPLE),
    ]
    for x0, label, color in entries:
        box(ax, (x0, 0.53), 0.15, 0.17, label, face=color + "20", edge=color, fontsize=11)
    for left, right in zip(entries[:-1], entries[1:]):
        arrow(ax, (left[0] + 0.15, 0.615), (right[0], 0.615), color=INK)
    box(ax, (0.26, 0.18), 0.20, 0.15, "Bernoulli\ncumulants", face=YELLOW + "30", edge=YELLOW, fontsize=12)
    box(ax, (0.54, 0.18), 0.20, 0.15, "Gaussian\nheat limit", face=RED + "20", edge=RED, fontsize=12)
    arrow(ax, (0.475, 0.53), (0.36, 0.33), color=ORANGE)
    arrow(ax, (0.835, 0.53), (0.64, 0.33), color=PURPLE)
    save_figure(fig, "01_theory_pipeline")
    stems.append("01_theory_pipeline")

    y = np.linspace(-6, 4, 900)
    fig, ax = plt.subplots(figsize=(12.5, 5.5))
    for alpha, color in zip([0.8, 1.5, 3.0, 6.0], COLORS):
        ax.plot(y, log_gamma_window(alpha, y), color=color, lw=2, label=rf"$\alpha={alpha}$")
    ax.set_xlabel("log coordinate $y$")
    ax.set_ylabel(r"$g_\alpha(y)$")
    ax.set_title("Log-Gamma packets across the shape coordinate")
    ax.grid(alpha=0.18)
    ax.legend(ncol=4)
```

```
    save_figure(fig, "02_shape_windows")
    stems.append("02_shape_windows")

    h = np.linspace(-5, 5, 220)
    eta = np.linspace(-5, 5, 220)
    hh, ee = np.meshgrid(h, eta, indexing="ij")
    mag = np.abs(cross_gamma_sech(1.0, 4.0, hh, ee))
    fig, ax = plt.subplots(figsize=(10.5, 6.2))
    image = ax.imshow(mag.T, origin="lower", extent=[h.min(), h.max(), eta.min(), eta.max()], aspect="auto", cmap="
     viridis")
    ax.set_xlabel("translation gap $h$")
    ax.set_ylabel(r"frequency gap $\eta$")
    ax.set_title(r"Cross-shape ambiguity $|A_{1,4}(h,\eta)|$")
    fig.colorbar(image, ax=ax, fraction=0.035, pad=0.02, label="magnitude")
    save_figure(fig, "03_cross_ambiguity")
    stems.append("03_cross_ambiguity")

    alphas = np.linspace(0.5, 6.0, 120)
    affinity = np.array([[shape_affinity(a, b) for b in alphas] for a in alphas])
    fig, ax = plt.subplots(figsize=(8.3, 6.6))
    image = ax.imshow(affinity, origin="lower", extent=[alphas.min(), alphas.max(), alphas.min(), alphas.max()],
     cmap="magma", vmin=0, vmax=1)
    ax.set_xlabel(r"$\beta$")
    ax.set_ylabel(r"$\alpha$")
    ax.set_title("Shape fidelity of two log-Gamma packets")
    fig.colorbar(image, ax=ax, fraction=0.045, pad=0.03, label="affinity")
    save_figure(fig, "04_shape_fidelity")
    stems.append("04_shape_fidelity")

    hline = np.linspace(-8, 8, 800)
    fig, ax = plt.subplots(figsize=(12.4, 5.3))
    for alpha, beta, color in [(1, 1, BLUE), (1, 4, TEAL), (2, 5, ORANGE), (5, 2, MAGENTA)]:
        ax.plot(hline, beta_logit_density(alpha, beta, hline), lw=2, color=color, label=rf"$(\alpha,\beta)=({alpha
     },{beta})$")
    ax.set_xlabel("translation gap $h$")
    ax.set_ylabel("density")
    ax.set_title("Beta-logit displacement packets")
    ax.grid(alpha=0.18)
    ax.legend(ncol=2)
    save_figure(fig, "05_beta_logit")
    stems.append("05_beta_logit")

    omega = np.linspace(-7, 7, 900)
    fig, ax = plt.subplots(figsize=(12.4, 5.3))
    for shape, color in zip([1, 2, 4, 8], COLORS):
        ax.plot(omega, meixner_density(shape, omega), lw=2, color=color, label=rf"shape $={shape}$")
    ax.set_xlabel(r"log-frequency $\omega$")
    ax.set_ylabel(r"$\rho_\alpha(\omega)$")
    ax.set_title("Meixner packet densities")
    ax.grid(alpha=0.18)
    ax.legend(ncol=4)
    save_figure(fig, "06_meixner_densities")
    stems.append("06_meixner_densities")

    hgrid = np.linspace(-5, 5, 180)
    egrid = np.linspace(-5, 5, 180)
    cross_density = cross_spectrogram_density(1.5, 3.5, hgrid, egrid)
    factorized = beta_logit_density(1.5, 3.5, hgrid)[:, None] * meixner_density(5.0, egrid)[None, :]
    fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.2))
    for ax, panel, title in zip(
        axes,
        [cross_density, factorized, np.abs(cross_density - factorized)],
        ["direct cross-spectrogram", "Beta-logit x Meixner", "absolute residual"],
    ):
        image = ax.imshow(panel.T, origin="lower", extent=[hgrid.min(), hgrid.max(), egrid.min(), egrid.max()],
     aspect="auto", cmap="viridis")
        ax.set_title(title, fontsize=10.5)
        ax.set_xlabel("$h$")
        ax.set_ylabel(r"$\eta$")
        fig.colorbar(image, ax=ax, fraction=0.045, pad=0.02)
    fig.suptitle("Exact factorization of cross-shape packet energy", fontsize=16, weight="bold")
```

```
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "07_cross_factorization")
    stems.append("07_cross_factorization")

    fig, ax = plt.subplots(figsize=(12.2, 5.2))
    for shape, color in zip([1, 2, 3, 4], COLORS):
        ax.plot(omega, meixner_density(shape, omega), color=color, lw=2, label=rf"$\rho_{shape}$")
    ax.set_xlim(-5, 5)
    ax.set_xlabel(r"$\omega$")
    ax.set_ylabel("density")
    ax.set_title("Shape ladder generated by the Gamma recurrence")
    ax.grid(alpha=0.18)
    ax.legend(ncol=4)
    save_figure(fig, "08_shape_ladder")
    stems.append("08_shape_ladder")

    dx = 0.01
    grid = np.arange(-16, 16 + dx, dx)
    rho_a = meixner_density(1.3, grid)
    rho_b = meixner_density(2.2, grid)
    conv = np.convolve(rho_a, rho_b, mode="same") * dx
    target = meixner_density(3.5, grid)
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.8))
    axes[0].plot(grid, conv, color=BLUE, lw=2, label=r"$\rho_{1.3}*\rho_{2.2}$")
    axes[0].plot(grid, target, color=ORANGE, lw=1.4, ls="--", label=r"$\rho_{3.5}$")
    axes[0].set_xlim(-6, 6)
    axes[0].set_title("Convolution semigroup")
    axes[0].legend()
    axes[0].grid(alpha=0.18)
    axes[1].semilogy(grid, np.maximum(np.abs(conv - target), 1e-16), color=RED, lw=1.4)
    axes[1].set_xlim(-6, 6)
    axes[1].set_title("numerical convolution residual")
    axes[1].grid(alpha=0.18)
    for ax in axes:
        ax.set_xlabel(r"$\omega$")
    fig.tight_layout()
    save_figure(fig, "09_convolution_semigroup")
    stems.append("09_convolution_semigroup")

    xpos = np.logspace(-3, 1, 700)
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.8))
    axes[0].loglog(xpos, levy_density_positive(xpos), color=PURPLE, lw=2)
    axes[0].set_xlabel("jump size $x$")
    axes[0].set_ylabel(r"$[2x\sinh(\pi x)]^{-1}$")
    axes[0].set_title("Levy density")
    axes[0].grid(alpha=0.18, which="both")
    xis = np.linspace(0, 6, 160)
    numeric_symbol = np.array([levy_symbol_numeric(float(x)) for x in xis])
    exact_symbol = -np.log(np.cosh(xis / 2.0))
    axes[1].plot(xis, numeric_symbol, color=BLUE, lw=2, label="Levy integral")
    axes[1].plot(xis, exact_symbol, color=ORANGE, lw=1.4, ls="--", label=r"$-\log\cosh(\xi/2)$")
    axes[1].set_xlabel(r"Fourier variable $\xi$")
    axes[1].set_title("Generator symbol")
    axes[1].grid(alpha=0.18)
    axes[1].legend()
    fig.tight_layout()
    save_figure(fig, "10_levy_generator")
    stems.append("10_levy_generator")

    orders = np.arange(2, 14, 2)
    shapes = [0.5, 1.0, 2.0, 4.0]
    fig, ax = plt.subplots(figsize=(12.2, 5.4))
    for shape, color in zip(shapes, COLORS):
        values = [bernoulli_even_cumulant(shape, int(order)) for order in orders]
        ax.plot(orders, values, marker="o", lw=2, color=color, label=rf"shape $={shape}$")
    ax.set_yscale("log")
    ax.set_xlabel("cumulant order")
    ax.set_ylabel("even cumulant")
    ax.set_title("Bernoulli-number cumulants of the Meixner packet semigroup")
    ax.grid(alpha=0.18, which="both")
    ax.legend(ncol=4)
```

```
    save_figure(fig, "11_bernoulli_cumulants")
    stems.append("11_bernoulli_cumulants")

    fig, ax = diagram_axes()
    ax.text(0.5, 0.93, "Exact polynomial packet splitter", ha="center", fontsize=21, weight="bold")
    box(ax, (0.08, 0.57), 0.20, 0.16, r"$p_{n,\alpha+\beta}(x+y)$", face=BLUE + "20", edge=BLUE, fontsize=14)
    for idx, r in enumerate(range(5)):
        y0 = 0.78 - 0.145 * idx
        box(ax, (0.43, y0), 0.18, 0.09, rf"$p_{{{r},\alpha}}(x)$", face=TEAL + "18", edge=TEAL, fontsize=10.5)
        box(ax, (0.72, y0), 0.18, 0.09, rf"$p_{{n-{r},\beta}}(y)$", face=ORANGE + "18", edge=ORANGE, fontsize=10.5)
        arrow(ax, (0.28, 0.65), (0.43, y0 + 0.045), color=INK, text=rf"$C_{{n,{r}}}$")
        arrow(ax, (0.61, y0 + 0.045), (0.72, y0 + 0.045), color=INK)
    ax.text(0.5, 0.08, r"$|C_{n,r}|^2$ is a Beta-binomial law and $\sum_r|C_{n,r}|^2=1$", ha="center", fontsize=12,
     color=MUTED)
    save_figure(fig, "12_packet_splitter")
    stems.append("12_packet_splitter")

    n = 12
    fig, ax = plt.subplots(figsize=(11.8, 5.0))
    for alpha, beta, color in [(1, 1, BLUE), (1, 4, TEAL), (2, 5, ORANGE), (5, 2, MAGENTA)]:
        coeffs = beta_binomial_coefficients(n, alpha, beta) ** 2
        ax.plot(np.arange(n + 1), coeffs, marker="o", lw=2, color=color, label=rf"$(\alpha,\beta)=({alpha},{beta})$"
     )
    ax.set_xlabel("child degree $r$")
    ax.set_ylabel(r"$|C_{n,r}^{\alpha,\beta}|^2$")
    ax.set_title("Beta-binomial energy partition")
    ax.grid(alpha=0.18)
    ax.legend(ncol=2)
    save_figure(fig, "13_beta_binomial_energy")
    stems.append("13_beta_binomial_energy")

    xvals = np.linspace(-4, 4, 500)
    y0 = 0.85
    degree = 4
    lhs = mp_orthonormal(5.0, degree, xvals + y0)
    rhs = mp_addition_rhs(2.0, 3.0, degree, xvals, np.full_like(xvals, y0))
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.8))
    axes[0].plot(xvals, lhs, color=BLUE, lw=2, label="parent packet")
    axes[0].plot(xvals, rhs, color=ORANGE, lw=1.3, ls="--", label="sum of child products")
    axes[0].set_title("Degree-four addition theorem")
    axes[0].legend()
    axes[0].grid(alpha=0.18)
    axes[1].plot(xvals, lhs - rhs, color=RED, lw=1.4)
    axes[1].set_title("pointwise residual")
    axes[1].grid(alpha=0.18)
    for ax in axes:
        ax.set_xlabel("$x$")
    fig.tight_layout()
    save_figure(fig, "14_addition_example")
    stems.append("14_addition_example")

    shapes3 = (1.0, 2.0, 3.0)
    degree3 = 8
    comps = list(compositions(degree3, 3))
    weights = np.array([dirichlet_multinomial_coefficient(c, shapes3) ** 2 for c in comps])
    coords = np.array([[c[0], c[1], c[2]] for c in comps])
    fig, ax = plt.subplots(figsize=(8.0, 6.3))
    scatter = ax.scatter(coords[:, 0] + 0.5 * coords[:, 1], np.sqrt(3) * coords[:, 1] / 2, c=weights, s=1600 *
     weights + 30, cmap="viridis", edgecolor=INK, linewidth=0.4)
    ax.set_aspect("equal")
    ax.set_xlabel(r"$r_1+r_2/2$")
    ax.set_ylabel(r"$\sqrt{3},r_2/2$")
    ax.set_title("Dirichlet-multinomial refinement on a three-child simplex")
    fig.colorbar(scatter, ax=ax, fraction=0.045, pad=0.03, label="squared coefficient")
    save_figure(fig, "15_multinomial_simplex")
    stems.append("15_multinomial_simplex")

    fig, ax = diagram_axes()
    ax.text(0.5, 0.93, "Coassociative packet refinement", ha="center", fontsize=21, weight="bold")
    box(ax, (0.40, 0.72), 0.20, 0.12, r"shape $\alpha+\beta+\gamma$", face=BLUE + "20", edge=BLUE, fontsize=12)
    box(ax, (0.14, 0.45), 0.22, 0.12, r"$(\alpha+\beta)+\gamma$", face=TEAL + "20", edge=TEAL, fontsize=12)
```

```
    box(ax, (0.64, 0.45), 0.22, 0.12, r"$\alpha+(\beta+\gamma)$", face=ORANGE + "20", edge=ORANGE, fontsize=12)
    box(ax, (0.36, 0.16), 0.28, 0.13, "same three-child coefficient\n" + r"$\sqrt{\frac{n!\prod(\alpha_j)_{r_j}}{(\sum\alpha_j)_n\prod r_j!}}$", face=MAGENTA + "18", edge=MAGENTA, fontsize=10.5)
    arrow(ax, (0.46, 0.72), (0.25, 0.57), color=TEAL)
    arrow(ax, (0.54, 0.72), (0.75, 0.57), color=ORANGE)
    arrow(ax, (0.25, 0.45), (0.43, 0.29), color=TEAL)
    arrow(ax, (0.75, 0.45), (0.57, 0.29), color=ORANGE)
    save_figure(fig, "16_coassociativity")
    stems.append("16_coassociativity")

    degrees = np.arange(0, 18)
    fig, ax = plt.subplots(figsize=(12.2, 5.2))
    for increment, color in zip([0.5, 1.0, 2.0, 5.0], COLORS):
        singular = [transfer_singular_value(2.0, increment, int(d)) for d in degrees]
        ax.plot(degrees, singular, marker="o", lw=2, color=color, label=rf"increment $={increment}$")
    ax.set_xlabel("polynomial packet degree")
    ax.set_ylabel("transfer singular value")
    ax.set_title("Exact singular spectrum of Meixner shape transfer")
    ax.grid(alpha=0.18)
    ax.legend(ncol=2)
    save_figure(fig, "17_transfer_singular_values")
    stems.append("17_transfer_singular_values")

    shape_grid = np.linspace(0.5, 8.0, 90)
    degree_grid = np.arange(0, 24)
    spectrum = np.array([[transfer_singular_value(float(s), 2.0, int(nv)) for nv in degree_grid] for s in shape_grid])
    fig, ax = plt.subplots(figsize=(10.2, 6.0))
    image = ax.imshow(spectrum, origin="lower", aspect="auto", extent=[degree_grid.min(), degree_grid.max(), shape_grid.min(), shape_grid.max()], cmap="magma", vmin=0, vmax=1)
    ax.set_xlabel("degree $n$")
    ax.set_ylabel(r"starting shape $\alpha$")
    ax.set_title("Shape-transfer contraction spectrum for increment two")
    fig.colorbar(image, ax=ax, fraction=0.038, pad=0.02, label="singular value")
    save_figure(fig, "18_transfer_spectrum")
    stems.append("18_transfer_spectrum")

    fig, ax = plt.subplots(figsize=(11.8, 5.2))
    for degree, color in zip([1, 2, 3, 5], COLORS):
        svals = [transfer_singular_value(1.0, float(t), degree) for t in np.linspace(0, 12, 300)]
        ax.plot(np.linspace(0, 12, 300), svals, lw=2, color=color, label=rf"degree $n={degree}$")
    ax.set_xlabel("shape increment $t$")
    ax.set_ylabel("conditional-transfer amplitude")
    ax.set_title("Polynomial martingale transfer")
    ax.grid(alpha=0.18)
    ax.legend(ncol=4)
    save_figure(fig, "19_martingale_transfer")
    stems.append("19_martingale_transfer")

    ugrid = np.linspace(-3.5, 3.5, 9)
    egrid2 = np.linspace(-3.5, 3.5, 9)
    fig, ax = plt.subplots(figsize=(8.0, 6.5))
    for u in ugrid:
        for eta0 in egrid2:
            magnitude = abs(coherent_overlap(2.0, 2.0, 0.0, 0.0, float(u), float(eta0)))
            ax.scatter(u, eta0, s=340 * magnitude + 8, color=BLUE, alpha=0.65, edgecolor=INK, linewidth=0.25)
    ax.set_xlabel(r"translation $u$")
    ax.set_ylabel(r"modulation $\eta$")
    ax.set_title("Log-Gamma coherent-state lattice")
    ax.grid(alpha=0.15)
    save_figure(fig, "20_coherent_lattice")
    stems.append("20_coherent_lattice")

    kernel_mag = np.abs(cross_gamma_sech(2.0, 2.0, hh, ee))
    fig, ax = plt.subplots(figsize=(9.5, 6.2))
    image = ax.imshow(kernel_mag.T, origin="lower", extent=[h.min(), h.max(), eta.min(), eta.max()], aspect="auto", cmap="plasma")
    ax.set_xlabel("translation difference")
    ax.set_ylabel("modulation difference")
    ax.set_title("Reproducing-kernel magnitude")
    fig.colorbar(image, ax=ax, fraction=0.04, pad=0.025, label="overlap magnitude")
```

```
    save_figure(fig, "21_rkhs_kernel")
    stems.append("21_rkhs_kernel")

    bkernel = berezin_kernel(2.0, hgrid, egrid)
    fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.2))
    axes[0].plot(hgrid, berezin_position_density(2.0, hgrid), color=TEAL, lw=2)
    axes[0].set_title("position marginal")
    axes[0].set_xlabel("$h$")
    axes[0].grid(alpha=0.18)
    axes[1].plot(egrid, berezin_frequency_density(2.0, egrid), color=ORANGE, lw=2)
    axes[1].set_title("frequency marginal")
    axes[1].set_xlabel(r"$\eta$")
    axes[1].grid(alpha=0.18)
    image = axes[2].imshow(bkernel.T, origin="lower", extent=[hgrid.min(), hgrid.max(), egrid.min(), egrid.max()],
     aspect="auto", cmap="viridis")
    axes[2].set_title("product kernel")
    axes[2].set_xlabel("$h$")
    axes[2].set_ylabel(r"$\eta$")
    fig.colorbar(image, ax=axes[2], fraction=0.045, pad=0.02)
    fig.suptitle("Exact Berezin-kernel factorization", fontsize=16, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "22_berezin_factorization")
    stems.append("22_berezin_factorization")

    xi = np.linspace(-5, 5, 180)
    xx = np.linspace(-5, 5, 180)
    multiplier = berezin_multiplier(2.0, xi, xx)
    fig, ax = plt.subplots(figsize=(9.5, 6.2))
    image = ax.imshow(multiplier.T, origin="lower", extent=[xi.min(), xi.max(), xx.min(), xx.max()], aspect="auto",
     cmap="magma", norm=matplotlib.colors.LogNorm(vmin=max(multiplier.min(), 1e-8), vmax=1))
    ax.set_xlabel(r"dual translation variable $\xi$")
    ax.set_ylabel("dual modulation variable $x$")
    ax.set_title("Strictly positive Berezin multiplier")
    fig.colorbar(image, ax=ax, fraction=0.04, pad=0.025, label="multiplier")
    save_figure(fig, "23_berezin_multiplier")
    stems.append("23_berezin_multiplier")

    z = np.linspace(-4.5, 4.5, 700)
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.8), sharey=True)
    for alpha, color in zip([2, 5, 20, 80], COLORS):
        axes[0].plot(z, rescaled_position_density(alpha, z), color=color, lw=1.8, label=rf"$\alpha={alpha}$")
        axes[1].plot(z, rescaled_frequency_density(alpha, z), color=color, lw=1.8, label=rf"$\alpha={alpha}$")
    for ax, title in zip(axes, ["rescaled position marginal", "rescaled frequency marginal"]):
        ax.plot(z, normal_pdf(z), color=INK, lw=2, ls="--", label="standard normal")
        ax.set_title(title)
        ax.set_xlabel("standardized coordinate")
        ax.grid(alpha=0.18)
    axes[0].set_ylabel("density")
    axes[1].legend(ncol=2, fontsize=8.5)
    fig.tight_layout()
    save_figure(fig, "24_gaussian_marginals")
    stems.append("24_gaussian_marginals")

    z2 = np.linspace(-3.5, 3.5, 180)
    pos = rescaled_position_density(20.0, z2)
    freq = rescaled_frequency_density(20.0, z2)
    joint = pos[:, None] * freq[None, :]
    gaussian = normal_pdf(z2)[:, None] * normal_pdf(z2)[None, :]
    fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.2))
    for ax, panel, title in zip(axes, [joint, gaussian, np.abs(joint - gaussian)], [r"rescaled $B_{20}$", "Gaussian
     heat kernel", "absolute residual"]):
        image = ax.imshow(panel.T, origin="lower", extent=[z2.min(), z2.max(), z2.min(), z2.max()], aspect="equal",
     cmap="viridis")
        ax.set_title(title, fontsize=10.5)
        ax.set_xlabel("scaled position")
        ax.set_ylabel("scaled frequency")
        fig.colorbar(image, ax=ax, fraction=0.045, pad=0.02)
    fig.suptitle("Gaussian fixed point of Berezin diffusion", fontsize=16, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "25_gaussian_heat_limit")
    stems.append("25_gaussian_heat_limit")
```

```
    fig, ax = diagram_axes()
    ax.text(0.5, 0.94, "Exact transfer hierarchy", ha="center", fontsize=21, weight="bold")
    box(ax, (0.05, 0.58), 0.18, 0.16, "arithmetic depth\n" + r"parameter $\alpha$", face=BLUE + "20", edge=BLUE, fontsize=12)
    box(ax, (0.30, 0.58), 0.18, 0.16, "Meixner Levy\nsemigroup", face=TEAL + "20", edge=TEAL, fontsize=12)
    box(ax, (0.55, 0.58), 0.18, 0.16, "polynomial packet\nrefinement", face=ORANGE + "20", edge=ORANGE, fontsize=12)
    box(ax, (0.78, 0.58), 0.17, 0.16, "coherent-state\nquantization", face=MAGENTA + "20", edge=MAGENTA, fontsize=12)
    for x0 in [0.23, 0.48, 0.73]:
        arrow(ax, (x0, 0.66), (x0 + 0.07, 0.66), color=INK)
    box(ax, (0.24, 0.20), 0.22, 0.15, "Bernoulli cumulants\nand Levy generator", face=YELLOW + "30", edge=YELLOW, fontsize=11)
    box(ax, (0.55, 0.20), 0.22, 0.15, "Berezin kernel\nand Gaussian limit", face=PURPLE + "18", edge=PURPLE, fontsize=11)
    arrow(ax, (0.39, 0.58), (0.35, 0.35), color=TEAL)
    arrow(ax, (0.865, 0.58), (0.66, 0.35), color=MAGENTA)
    save_figure(fig, "26_transfer_hierarchy")
    stems.append("26_transfer_hierarchy")

    alpha_values = np.linspace(0.4, 8.0, 300)
    metric_shape = 0.25 * polygamma(1, alpha_values)
    metric_translation = alpha_values / 4.0
    metric_modulation = polygamma(1, alpha_values)
    fig, ax = plt.subplots(figsize=(12.2, 5.2))
    ax.plot(alpha_values, metric_shape, color=BLUE, lw=2, label=r"shape $g_{\alpha\alpha}$")
    ax.plot(alpha_values, metric_translation, color=TEAL, lw=2, label=r"translation $g_{uu}$")
    ax.plot(alpha_values, metric_modulation, color=ORANGE, lw=2, label=r"modulation $g_{\eta\eta}$")
    ax.set_xlabel(r"shape $\alpha$")
    ax.set_ylabel("metric coefficient")
    ax.set_title("Three local scales of the generated packet geometry")
    ax.set_yscale("log")
    ax.grid(alpha=0.18, which="both")
    ax.legend(ncol=3)
    save_figure(fig, "27_shape_geometry")
    stems.append("27_shape_geometry")

    fig, ax = diagram_axes()
    ax.text(0.5, 0.94, "Function-family map", ha="center", fontsize=21, weight="bold")
    nodes = [
        (0.06, 0.63, "cross-Gamma--sech\nambiguity", BLUE),
        (0.30, 0.63, "Meixner\nLevy semigroup", TEAL),
        (0.54, 0.63, "Meixner--Pollaczek\npacket coproduct", ORANGE),
        (0.78, 0.63, "Gamma--sech\nBerezin kernel", MAGENTA),
        (0.18, 0.25, "Beta-logit\ndisplacement law", PURPLE),
        (0.43, 0.25, "Bernoulli\ncumulants", YELLOW),
        (0.68, 0.25, "Gaussian\nheat fixed point", RED),
    ]
    for x0, y0, label, color in nodes:
        box(ax, (x0, y0), 0.17, 0.14, label, face=color + "20", edge=color, fontsize=10.5)
    for start, end in [((0.23, 0.70), (0.30, 0.70)), ((0.47, 0.70), (0.54, 0.70)), ((0.71, 0.70), (0.78, 0.70)),
                       ((0.145, 0.63), (0.265, 0.39)), ((0.385, 0.63), (0.515, 0.39)), ((0.865, 0.63), (0.765, 0.39))]:
        arrow(ax, start, end, color=INK)
    save_figure(fig, "28_function_family_map")
    stems.append("28_function_family_map")

    return stems


# ---------------------------------------------------------------------------
# Verification and tables
# ---------------------------------------------------------------------------


@dataclass
class Check:
    name: str
    passed: bool
    residual: float
    note: str

```

```

def run_checks() -> tuple[list[Check], dict[str, float | int | str]]:
    checks: list[Check] = []

    def add(name: str, residual: float, tolerance: float, note: str) -> None:
        checks.append(Check(name, bool(residual <= tolerance), float(residual), note))

    add("log-Gamma normalization", abs(quad(lambda y: float(log_gamma_window(2.3, y) ** 2), -30, 10)[0] - 1.0), 2e
    -11, "L2 norm")

    cross_exact = complex(cross_gamma_sech(1.4, 3.1, 0.75, -0.8))
    cross_num = cross_gamma_numeric(1.4, 3.1, 0.75, -0.8)
    add("cross-shape ambiguity", abs(cross_exact - cross_num), 3e-10, "closed form against quadrature")

    add("shape affinity strictness", max(0.0, shape_affinity(1.2, 4.7) - 1.0), 1e-14, "Cauchy and Gamma log-
    convexity")

    beta_integral = quad(lambda h: float(beta_logit_density(1.3, 2.7, h)), -30, 30, epsabs=2e-12)[0]
    add("Beta-logit normalization", abs(beta_integral - 1.0), 3e-10, "displacement marginal")

    meixner_integral = quad(lambda w: float(meixner_density(3.4, w)), -30, 30, epsabs=2e-12)[0]
    add("Meixner normalization", abs(meixner_integral - 1.0), 3e-10, "spectral marginal")

    htest = np.linspace(-3, 3, 51)
    etest = np.linspace(-3, 3, 51)
    direct = cross_spectrogram_density(1.3, 2.7, htest, etest)
    factor = beta_logit_density(1.3, 2.7, htest)[:, None] * meixner_density(4.0, etest)[None, :]
    add("cross-spectrogram factorization", np.max(np.abs(direct - factor)), 2e-13, "Beta-logit times Meixner")

    conv_grid = np.linspace(-18, 18, 7201)
    dx = conv_grid[1] - conv_grid[0]
    conv = fftconvolve(meixner_density(1.2, conv_grid), meixner_density(2.3, conv_grid), mode="same") * dx
    target = meixner_density(3.5, conv_grid)
    mask = np.abs(conv_grid) < 7
    add("Meixner convolution semigroup", np.max(np.abs(conv[mask] - target[mask])), 3e-4, "FFT-free grid convolution
    ")

    recurrence_residual = np.max(np.abs(meixner_density(3.2, etest) - density_shape_recurrence(1.2, etest)))
    add("shape recurrence", recurrence_residual, 2e-13, "shape-plus-two ladder")

    levy_points = [0.4, 1.1, 2.3]
    levy_residual = max(abs(levy_symbol_numeric(x) + math.log(math.cosh(x / 2.0))) for x in levy_points)
    add("Levy-Khintchine symbol", levy_residual, 3e-11, "integral against 1/(x sinh pi x)")

    add("Bernoulli variance", abs(bernoulli_even_cumulant(2.7, 2) - 2.7 / 4.0), 2e-14, "second cumulant")
    add("Bernoulli fourth cumulant", abs(bernoulli_even_cumulant(2.7, 4) - 2.7 / 8.0), 1e-11, "fourth cumulant")

    coeffs = beta_binomial_coefficients(12, 1.7, 3.4)
    add("Beta-binomial energy partition", abs(float(np.sum(coeffs * coeffs)) - 1.0), 2e-13, "sum of squared packet
    coefficients")

    x = np.linspace(-2.5, 2.5, 41)
    y = np.full_like(x, 0.73)
    lhs = mp_orthonormal(5.1, 6, x + y)
    rhs = mp_addition_rhs(1.7, 3.4, 6, x, y)
    add("normalized addition theorem", np.max(np.abs(lhs - rhs)), 3e-12, "two-child packet reconstruction")

    shapes = (0.9, 1.6, 2.4)
    points = (0.3, -0.8, 1.1)
    lhs_multi = float(mp_orthonormal(sum(shapes), 5, sum(points)))
    rhs_multi = mp_multinomial_rhs(shapes, 5, points)
    add("multinomial packet theorem", abs(lhs_multi - rhs_multi), 5e-12, "three-child reconstruction")

    coeff_sum = sum(dirichlet_multinomial_coefficient(c, shapes) ** 2 for c in compositions(7, 3))
    add("Dirichlet-multinomial energy", abs(coeff_sum - 1.0), 3e-13, "coassociative coefficient norm")

    transfer_points = [-1.3, 0.0, 1.8]
    residuals = []
    for degree in [0, 1, 2, 3]:
        sigma = transfer_singular_value(1.8, 1.1, degree)
        for xv in transfer_points:
```

```
            integral = transfer_integral(1.8, 1.1, degree, xv)
            target_value = sigma * float(mp_orthonormal(1.8, degree, xv))
            residuals.append(abs(integral - target_value))
    add("conditional transfer spectrum", max(residuals), 5e-8, "Meixner increment integral")

    cocycle = abs(
        transfer_singular_value(1.3, 0.8, 5)
        * transfer_singular_value(2.1, 1.4, 5)
        - transfer_singular_value(1.3, 2.2, 5)
    )
    add("transfer cocycle", cocycle, 3e-14, "singular values telescope")

    overlap_closed = coherent_overlap(1.4, 2.6, -0.3, 0.7, 0.8, -0.4)
    overlap_num = coherent_overlap_numeric(1.4, 2.6, -0.3, 0.7, 0.8, -0.4)
    add("coherent overlap", abs(overlap_closed - overlap_num), 4e-10, "Weyl phase and cross-shape kernel")

    pos_norm = quad(lambda h: float(berezin_position_density(2.4, h)), -30, 30, epsabs=2e-12)[0]
    freq_norm = quad(lambda w: float(berezin_frequency_density(2.4, w)), -30, 30, epsabs=2e-12)[0]
    add("Berezin position normalization", abs(pos_norm - 1.0), 3e-10, "position factor")
    add("Berezin frequency normalization", abs(freq_norm - 1.0), 3e-10, "frequency factor")

    position_variance = quad(lambda h: float(h * h * berezin_position_density(2.4, h)), -30, 30, epsabs=2e-11)[0]
    frequency_variance = quad(lambda w: float(w * w * berezin_frequency_density(2.4, w)), -30, 30, epsabs=2e-11)[0]
    add("Berezin position variance", abs(position_variance - 2.0 * float(polygamma(1, 2.4))), 3e-9, "twice the
     trigamma")
    add("Berezin frequency variance", abs(frequency_variance - 2.4 / 2.0), 3e-9, "shape divided by two")

    z = np.linspace(-7, 7, 5001)
    gaussian_cdf = ndtr(z)
    pos_density = rescaled_position_density(80.0, z)
    freq_density = rescaled_frequency_density(80.0, z)
    dz = z[1] - z[0]
    pos_cdf = np.cumsum(pos_density) * dz
    freq_cdf = np.cumsum(freq_density) * dz
    pos_cdf /= pos_cdf[-1]
    freq_cdf /= freq_cdf[-1]
    add("Gaussian position limit", np.max(np.abs(pos_cdf - gaussian_cdf)), 0.02, "Kolmogorov distance at alpha=80")
    add("Gaussian frequency limit", np.max(np.abs(freq_cdf - gaussian_cdf)), 0.02, "Kolmogorov distance at alpha=80"
     )

    passed = sum(check.passed for check in checks)
    results = {
        "checks_passed": passed,
        "checks_total": len(checks),
        "max_residual": max(check.residual for check in checks),
        "cross_shape_example_real": float(np.real(cross_exact)),
        "cross_shape_example_imag": float(np.imag(cross_exact)),
        "shape_affinity_1_4": shape_affinity(1.0, 4.0),
        "levy_symbol_at_1": levy_symbol_numeric(1.0),
        "beta_binomial_norm": float(np.sum(coeffs * coeffs)),
        "transfer_degree_5": transfer_singular_value(2.0, 2.0, 5),
        "berezin_position_variance_alpha_2_4": position_variance,
        "berezin_frequency_variance_alpha_2_4": frequency_variance,
        "figure_pairs": 28,
    }
    return checks, results


def write_outputs(stems: list[str], checks: list[Check], results: dict[str, float | int | str]) -> None:
    with (RESULTS / "checks.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=["name", "passed", "residual", "note"])
        writer.writeheader()
        for check in checks:
            writer.writerow({"name": check.name, "passed": check.passed, "residual": f"{check.residual:.16e}", "note
     ": check.note})

    (RESULTS / "results.json").write_text(json.dumps(results, indent=2, sort_keys=True) + "\n", encoding="utf-8")

    with (TABLES / "cumulants.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.writer(handle)
        writer.writerow(["shape", "kappa2", "kappa4", "kappa6", "kappa8"])
```

```
        for shape in [0.5, 1.0, 2.0, 4.0]:
            writer.writerow([shape] + [bernoulli_even_cumulant(shape, order) for order in [2, 4, 6, 8]])

    with (TABLES / "transfer_spectrum.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.writer(handle)
        writer.writerow(["degree", "increment_0_5", "increment_1", "increment_2", "increment_5"])
        for degree in range(0, 13):
            writer.writerow([degree] + [transfer_singular_value(2.0, increment, degree) for increment in [0.5, 1.0,
 2.0, 5.0]])

    def latex_escape(text: str) -> str:
        replacements = {
            "\\": r"\textbackslash{}",
            "&": r"\&",
            "%": r"\%",
            "$": r"\$",
            "#": r"\#",
            "_": r"\_",
            "{": r"\{",
            "}": r"\}",
        }
        return "".join(replacements.get(char, char) for char in text)

    row_break = r"\\"
    check_rows = []
    for check in checks:
        status = "pass" if check.passed else "FAIL"
        check_rows.append(
            f"{latex_escape(check.name)} & {status} & {check.residual:.3e} & {latex_escape(check.note)} {row_break}"
        )
    checks_tex = (
        "\\begin{longtable}{L{0.31\\textwidth}L{0.08\\textwidth}L{0.13\\textwidth}L{0.37\\textwidth}}\n"
        "\\toprule\n"
        f"Check & Status & Residual & Note {row_break}\n"
        "\\midrule\n"
        "\\endfirsthead\n"
        "\\toprule\n"
        f"Check & Status & Residual & Note {row_break}\n"
        "\\midrule\n"
        "\\endhead\n"
        + "\n".join(check_rows)
        + "\n\\bottomrule\n\\end{longtable}\n"
    )
    (GENERATED / "checks.tex").write_text(checks_tex, encoding="utf-8")

    (GENERATED / "headline.tex").write_text(
        "\\begin{center}\n"
        "\\fbox{\\begin{minipage}{0.83\\textwidth}\n"
        f"\\textbf{{Built-in verification.}} {results['checks_passed']} of {results['checks_total']} checks passed.
 "
        f"The release contains {results['figure_pairs']} vector-PDF and PNG figure pairs.\n"
        "\\end{minipage}}\n"
        "\\end{center}\n",
        encoding="utf-8",
    )

    figure_lines = [f"{index}. `{stem}.pdf` and `{stem}.png`" for index, stem in enumerate(stems, start=1)]
    (ROOT / "Projector_Generated_Special_Functions_VI_FIGURE_INDEX.md").write_text(
        "# Figure index\n\n" + "\n".join(figure_lines) + "\n",
        encoding="utf-8",
    )

    manifest = {
        "title": "Projector-Generated Special Functions VI",
        "author": "Komorebi P.",
        "figure_pairs": len(stems),
        "checks_passed": results["checks_passed"],
        "checks_total": results["checks_total"],
        "python_generator": "projector_generated_special_functions_vi.py",
    }
    (ROOT / "Projector_Generated_Special_Functions_VI_MANIFEST.json").write_text(
```

```
        json.dumps(manifest, indent=2, sort_keys=True) + "\n",
        encoding="utf-8",
    )


def main() -> None:
    stems = generate_figures()
    checks, results = run_checks()
    write_outputs(stems, checks, results)
    print(json.dumps(results, indent=2, sort_keys=True))
    if results["checks_passed"] != results["checks_total"]:
        raise SystemExit("one or more built-in checks failed")


if __name__ == "__main__":
    main()
```

Chapter D

Reproduction

From the source directory, run

```
python projector_generated_special_functions_vi.py
pytest -q
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_VI.tex
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_VI.tex
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_VI.tex
```

Chapter E

Bibliographic note

Bibliography

[1] NIST Digital Library of Mathematical Functions. Chapter 5, Gamma Function. `https://dlmf.nist.gov/5`.

[2] NIST Digital Library of Mathematical Functions. Sections 18.19 and 18.23, Meixner–Pollaczek polynomials and generating functions. `https://dlmf.nist.gov/18.23`.

[3] R. Koekoek, P. A. Lesky, and R. F. Swarttouw. *Hypergeometric Orthogonal Polynomials and Their q-Analogues*. Springer, 2010.

[4] W. Schoutens. *Stochastic Processes and Orthogonal Polynomials*. Springer, 2000.

[5] K. Sato. *Lévy Processes and Infinitely Divisible Distributions*. Cambridge University Press, 1999.

[6] G. B. Folland. *Harmonic Analysis in Phase Space*. Princeton University Press, 1989.

[7] F. A. Berezin. General concept of quantization. *Communications in Mathematical Physics*, 40:153–174, 1975.

[8] A. Perelomov. *Generalized Coherent States and Their Applications*. Springer, 1986.

[9] S. Wei, Y. Zhai, Q. Lu, W. Yang, P. Gao, C. Wei, J. Song, F. Nori, T. Xin, and G. Long. The Riemann Hypothesis manifested in dynamical quantum phase transitions. *Nature Communications*, article in press, 2026. DOI: 10.1038/s41467-026-74935-8.

[10] Komorebi P. *Projector-Generated Special Functions V: Log-Gamma Gabor Packets, Gamma–Sech Ambiguity Kernels, Meixner–Pollaczek Spectra, and* $\mathrm{SU}(1,1)$ *Duality*. Preceding paper in this series.